

MyCaTDD · 方法论系列

CaTDD 实战

CaTDD in Action · 评论即验证设计 — LLM 时代的实战手册

方法作者：EnigmaWU

目录

第 1 章 · 定义核心概念	5
什么是 CaTDD ?	5
按类别划分的 Method Prompt	6
四层架构	7
反馈循环	9
设计骨架合约	9
优先级框架	9
针对特定上下文的优先级调整	11
风险驱动的优先级调整	12
US/AC/TC 合约	12
TDD 红→绿循环 (CaTDD 风格)	13
质量关卡 (Quality Gates)	14
Fast-Fail Six	15
测试组织策略	15
实现跟踪模板	16
设计即活合约	17
从概念到行动	17
第 2 章 · chatVibeCoding	18
什么是 Vibe Coding ?	18
CaTDD 交互的两种模式	18
何时使用 VibeCoding	18
何时切换到 SpecCoding	19
VibeCoding 如何配合 methodPrompts 工作	19
VibeCoding workflow	20
VibeCoding vs SpecCoding : 开发者的决策	21
Method Prompt 作为聊天上下文	22
VibeCoding 的模型层级策略	23
常见的 VibeCoding 模式	23
VibeCoding 中的两个设计阶段	24
带有实时反馈的 VibeCoding	25
CodeAgent 在 VibeCoding 中的角色	26
VibeCoding 反模式	26
从 VibeCoding 到 SpecCoding : 桥梁	27
第 3 章 · callSlashCommands	28
什么是 Slash 命令 ?	28
三大命令族系	28
通用命令模板	29

四种命令流程与 HarnessKits 套件	30
P0 FuncTestsFlow —— 第一个需要学习的流程	30
Slash 命令中的 RED→GREEN 纪律	33
P1-DesignTestsFlow 和 P2-QualityTestsFlow	34
Px-SpecFlow —— 主编排流程	36
命令的模型层级指引	43
命令中的思维链模式	44
SpecFlow 工作制品	45
命令模板即契约	46
冲突保护规则	47
执行模式与 ONE-MORE-THING 停止规则	47
实践中的命令调用	47
何时使用哪个流程	48
子 Agent 模式	49
从命令到自动化	49
第 4 章 · asyncCodeAgent	50
什么是 Code Agent ?	50
两种代码 Agent	50
CaTDD 原生契约	51
Agent 工作流检查清单	51
Agent 故障排除：六大常见问题	54
Agent 的"该做与不该做"契约	55
追踪收集与反思	56
utCodeAgentCLI 架构	57
specCodeAgentCLI 架构	60
异步代码 Agent 工作流	61
Agent 与开发者检查点的关系	62
Agent 技能 vs 代码 Agent	62
代码 Agent 的未来	63
从自动化到集成	63
第 5 章 · applyClassicSWE	64
软件工程知识手册	64
LLM 时代改变了一切——也什么都没改变	64
LLM 时代的 TDD	64
LLM 时代的 BDD	66
LLM 时代的 DDD	68
CaTDD：TDD + BDD + DDD 的融合	70
LLM 作为知识手册阅读器	71
BDD → CaTDD：从 Feature 文件到注释链	71
DDD → CaTDD：从领域模型到注释骨架	72

P0→P3 框架作为战略 DDD	73
LLM 时代的鲜活文档	74
应用知识手册：一个完整示例	75
知识手册应用于实践	76
结语：经典 SWE 的三角	77
第 6 章 · beyondXyzSpec	78
规格驱动开发现状	78
GitHub Spec Kit — 以文档为中心的规格驱动开发	78
CaTDD 从 Spec Kit 采纳的内容	79
OpenSpec — 以变更为中心的规格驱动开发	80
CaTDD Px-SpecFlow — 以用户故事为中心的规格编码	81
UT 流程 — CaTDD 的嵌入式验证引擎	83
三向对比	85
为什么 CaTDD 是 BEYOND 的	86

定义核心概念

devBooks/ActionZH/01_defineConcepts.md

什么是 CaTDD ?

CaTDD —— Comment-alive Test-Driven Development (评论即设计测试驱动开发) —— 是由 EnigmaWU 发明、自 2023 年 10 月开始实践的一套方法论。它通过将注释变为 LLM 可解析、可保留、可更新的一等工件，重新定义了软件开发中"设计"的含义。

核心口号道出了其本质：

Comments is Verification Design. LLM Generates Code. Iterate Forward Together.

在传统开发中，设计存在于独立的文档中，一旦代码发生变化，这些文档就会过时。在 CaTDD 中，设计以结构化注释的形式存在于测试文件中，随代码一起演进。注释不是文档——它们就是验证规约。

"评论即设计" (Comment-alive) 的含义

"评论即设计"是 CaTDD 的定义性创新，它有四个维度：

1. 设计细节以结构化注释的形式存在于测试文件和源文件中。你不需要维护单独的规约文档、架构文档和测试计划。验证设计就嵌入在代码所在之处。
2. 注释随代码一起演进，而不是在会过时的独立文档中。当你重构生产代码时，你同时在同一个文件中更新注释。设计意图和实现通过构造方式保持同步。
3. 注释是 LLM 可以解析和更新的一等工件。阅读 CaTDD 测试文件的 LLM 能看到结构化的 `@[US]`、`@[AC]`、`@[TC]` 标记，它可以追踪、验证这些标记并从中生成代码。这就是 CaTDD 被称为"LLM 友好的 TDD"的原因。
4. US/AC/TC 格式在人类意图和机器可执行测试之间架起了桥梁。用户故事 (User Story) 表达业务价值。验收标准 (Acceptance Criteria) 使故事可测试。测试用例 (Test Case) 将验收标准连接到具体的断言。这条链路从人类需求直达机器验证，中间没有缺口。

"TDD" 部分

CaTDD 继承了传统 TDD 的红→绿→重构循环，但在编码之前增加了结构化设计：

传统 TDD	CaTDD
先写一个失败的测试	先写结构化注释设计 (US/AC/TC)，再写失败的测试
实现代码使其通过	实现最小生产代码使其通过
重构	同时重构测试代码和生产代码
重复	标记 TC 状态 (○→●→●)，推进到下一个

传统 TDD	CaTDD
没有显式的设计工件	设计与测试共存于同一文件中

CaTDD 不是 TDD 的替代品。它是内置了设计结构、可追溯性和 LLM 可读性的 TDD。

按类别划分的 Method Prompt

除了主 method prompt 之外，CaTDD 提供了 15 个按类别划分的 prompt——每个测试类别一个。这些深入指导文件（CaTDD_methodPrompt4Cat-*.md）在 CodeAgent 针对特定类别工作时给予指引：

类别	Prompt 文件	核心指引
Typical	CaTDD_methodPrompt4Cat-Typical.md	核心正常路径设计，每个测试一个行为， ≤3 个关键断言
Edge	CaTDD_methodPrompt4Cat-Edge.md	边界值、模式变体，每个边界一个测试以便诊断
Misuse	CaTDD_methodPrompt4Cat-Misuse.md	API 合约违规、错误预防、非法调用序列
Fault	CaTDD_methodPrompt4Cat-Fault.md	外部故障、恢复、优雅降级
State	CaTDD_methodPrompt4Cat-State.md	生命周期转换、不变量、非法转换拒绝
Capability	CaTDD_methodPrompt4Cat-Capability.md	设计的能力边界、极限与组件职责
Interaction	CaTDD_methodPrompt4Cat-Interaction.md	协作者顺序、交接契约、编排顺序
Concurrency	CaTDD_methodPrompt4Cat-Concurrency.md	线程安全、并发访问下的顺序、竞态/死锁自由
Performance	CaTDD_methodPrompt4Cat-Performance.md	SLO 验证、延迟/吞吐/资源预算
Robust	CaTDD_methodPrompt4Cat-Robust.md	压力、重复、降级条件、长期运行稳定性
Compatibility	CaTDD_methodPrompt4Cat-Compatibility.md	版本、平台、协议、schema、工具链与集成接口
Configuration	CaTDD_methodPrompt4Cat-Configuration.md	默认值、优先级、功能开关、环境变量与非法配置
Diagnosis	CaTDD_methodPrompt4Cat-Diagnosis.md	可观测性、可调试性、可操作证据、失败可解释性
Security	CaTDD_methodPrompt4Cat-Security.md	威胁模型下的保护属性、信任边界、机密处理
Demo/Example	CaTDD_methodPrompt4Cat-DemoExample.md	教程、文档、最佳实践示例

每个类别 prompt 都定义了：Position（在优先级框架中的位置）、Use When（适用条件）、Do Not Use When（何时应将场景移至其他类别）、TestPointsInMind（该类别应考虑的测试点）、Design Skeleton（合约骨架）以及 Checklist（质量验证清单）。其中 10 个（4 个 P0 类别加上 State、Capability、Interaction、Concurrency、Diagnosis、Security）还保留了 Design Focus、US/AC/TC Pattern、Naming Examples 和 Common Mistakes 指引；其余质量/附加文件（Performance、Robust、Compatibility、Configuration、Demo/Example）保持精简，通过 TestPointsInMind 引导发现。

这些 prompt 是 LLM 针对每个类别的"风格指南"。当 CodeAgent 设计一个 Typical 设计骨架时，它会读取 `CaTDD_methodPrompt4Cat-Typical.md` 来应用正确的模式、术语和约束。当它将草案分类到类别中时，它会检查"Do Not Use When"规则以避免错误分类。

四层架构

MyCaTDD 将 CaTDD 方法论组织为四个层级，每个层级职责清晰：

[1] <code>methodPrompts</code>	- 方法源头 CaTDD 语义、类别含义、US/AC/TC 骨架规则、状态纪律、模板
[2] <code>slashCommands</code>	- 命令流程 可移植的 <code>UT_*</code> 、 <code>SPEC_*</code> 、 <code>HARNESS_*</code> 命令 输入/输出传递、工具中立的执行
[3] <code>codeAgents</code>	- 智能执行 <code>utCodeAgentCLI</code> （单元测试）、 <code>specCodeAgentCLI</code> 规划、追踪收集、反思循环
[4] <code>agentSkills</code>	- 可复用能力包 <code>Comment-alive</code> TDD 技能、 <code>SpecCoding</code> 技能 打包供 Copilot、Cline 及其他 agent 使用

第 1 层：methodPrompts — 真理之源

`methodPrompts/` 是 CaTDD 的规范性定义。下游的一切都源自这一层。如果方法论有变化，这一层最先变化。

它包含：

- **CaTDD_methodPrompt.md** — 主方法入口与稳定的 CaTDD 合约：设计骨架形态、类别语义、强制可追溯性、测试点发现、工作流与默认执行顺序。
- **CaTDD_methodPrompt-*.md** — 方法子主题 prompt，承载主入口引用的深入材料：类别语义、测试点发现、工作流、测试结构、文件命名、agent 工作流、故障排查与示例。
- **CaTDD_methodPrompt4Cat-*.md** — 按类别划分的 method prompt。15 个测试类别（Typical、Edge、Misuse、Fault、State、Capability、Interaction、Concurrency、Performance、Robust、Compatibility、Configuration、Diagnosis、Security、Demo/Example）各自都有专门的深入 prompt，CodeAgent 在该类别下工作时可以参考。
- **CaTDD_designAndImplTemplate.cxx/.ts/.py/.go** — 针对 C++、TypeScript、Python 和 Go 的语言实现模板，展示完整的 CaTDD 文件结构。每个模板都遵循相同的 OVERVIEW → DESIGN → IMPLEMENTATION → TODO 结构；没有任何语言是方法上的硬性要求。
- 独立用户指南 — `README_UserGuide.md` 和 `README_UserGuide_ZH.md` 解释了如何使用 method prompt、谁使用它们、何时应用以及放在哪里。它们的存在使得用户无需阅读整个仓库，仅使用此目录即可上手。

第 2 层：slashCommands — 命令化层

`slashCommands/` 将稳定的方法步骤转化为可移植的命令。它是方法语义与 CodeAgent 调用界面（Copilot、Cline、Continue、`utCodeAgentCLI`）之间的连接器。

核心原则：slashCommands 不重新定义方法语义。它取 methodPrompts 定义的内容，将其包装为可执行的命令单元。当命令与 methodPrompts 冲突时，以 methodPrompts 为准。

该层包含：

- **flows/** 下的流程文档 — P0-FuncTestsFlow、P1-DesignTestsFlow、P2-QualityTestsFlow 和 Px-SpecFlow。每个流程定义了可重复的命令序列，包含入口点、关卡和循环返回路径。
- **kits/** 下的套件文档 — Px-HarnessKits 将运维性的 **HARNESS_*** 工具点命令分组，用于维护 CaTDD 源码、适配器、执行、诊断与补丁回写安全，不要求严格的生命周期顺序。
- **命令模板** — **UT_slashCommandTemplate.md** 和 **SPEC_slashCommandTemplate.md** 确保每个命令遵循相同的 Command Header / CoT Pattern / WHO / WHAT / WHEN / WHERE / WHY / HOW / Input Contract / Output Contract / CodeAgent Compatibility 结构；SPEC 模板额外包含 Subagent Recommendation。
- **commands/** 下的具体命令文件 — 三个命令族（**UT_***、**SPEC_***、**HARNESS_***）按流程与套件分组。每个命令文件告诉 CodeAgent 具体做什么：读取什么、产出什么、保留什么，以及接下来执行什么命令。

第3层：codeAgents — 执行层

codeAgents/ 包含 CaTDD 原生的 CLI agent 概念。目前这些是设计文档和架构规约——可运行的实现正在进行中。

定义了两个 agent：

- **utCodeAgentCLI** — 单元测试代码 agent。它接收开发者目标作为输入，从 CaTDD 方法约束出发进行规划，调用标准化的 slash command 步骤，收集追踪信息，反思结果，并将可复用的模式反馈回方法层和命令层。它保留 CaTDD 的设计骨架合约、US/AC/TC 可追溯性、类别分类以及 RED/GREEN 状态纪律。
- **specCodeAgentCLI** — 规约代码 agent。它基于 Px-SpecFlow 编排模块级的 SpecCoding 流程，从输入到输出。它复用 utCodeAgentCLI 的单元测试能力，组织从规约意图到可执行检查的场景级验证。它保持规约流程、验证检查点和实现结果之间的可追溯性。

这两个 agent 共同构成一条流水线：spec → ut → results。

第4层：agentSkills — 打包层

agentSkills/ 将 CaTDD 和 SpecCoding 打包为任何 CodeAgent 都可以消费的可复用技能。编写了两个技能：

1. **comment-alive-test-driven-development** — CaTDD 测试方法论打包为技能。它包含 WHO/WHAT/WHEN/WHERE/WHY 部分、分阶段执行指令、输入/输出合约、约束和验证规则。当开发者对 CodeAgent 说"使用 CaTDD"时，此技能为 agent 提供所需的一切。
2. **user-story-centered-spec-coding** — SpecCoding 生命周期编排打包为技能。它涵盖完整的故事生命周期：pendingNews → todoUS → doingUS → doneUS，其中 abortUS 保留不安全的活跃故事以供后续分析，CaTDD 作为默认的单元测试方法。

打包脚本 **makeSkill.sh** 通过从 methodPrompts 和 slashCommands 复制引用，生成自包含的可分发包。编写的源文件是持久资产；生成的包是构建输出。

反馈循环

这四个层级不是单向流水线。它们形成一个双向改进循环：



- methodPrompts → slashCommands：稳定的方法步骤被命令化为 slash command。
- methodPrompts → codeAgents：Agent 行为受到方法语义的约束。
- slashCommands → codeAgents：Agent 调用标准化的命令步骤。
- slashCommands → methodPrompts：命令执行揭示方法缺口——方法得到改进。
- codeAgents → slashCommands：Agent 反思识别可复用的命令模式——slash command 不断增长。
- codeAgents → methodPrompts：执行经验反馈到方法论的改进中。

这个循环确保 CaTDD 从实际使用中演进，而非从理论设计中产生。每个下游层既消费又改进上游层。

设计骨架合约

在 CaTDD 中，“设计”不是 UML 图或 Word 文档。它是测试文件内部的可复用注释骨架。每个骨架按以下方式组织：

- Class：优先级族 — P0 Functional、P1 Design、P2 Quality、P3 Addons
- Category：具体的验证角度 — Typical、Edge、Misuse、Fault、State、Capability、Interaction、Concurrency、Performance、Robust、Compatibility、Configuration、Diagnosis、Security、Demo/Example

每个骨架保留以下最小形态：

```

//=====
// [Class] / [Category] Design Skeleton
//=====
// @[SUT]: [声明的 SUT，与文件 OVERVIEW 一致]
// @[TestLevel]: UnitTesting (或 SysTesting / UserTesting)
// @[Class]: P0 Functional / ValidFunc
// @[Category]: Typical
// @[Intent]: 此类别对该组件要验证什么
// @[UseWhen]: 何时使用此类别
// @[AvoidWhen]: 何时将场景移至其他类别
// @[US]: 此类别覆盖的 User Story ID
// @[AC]: 此类别覆盖的 Acceptance Criteria ID
// @[TC]: Test Cases、状态及预期的 TDD 下一步动作
//=====
  
```

这个骨架是开发者和 CodeAgent 共同遵守的“合约”。开发者用验证意图填充骨架。CodeAgent 读取骨架并生成满足该意图的测试代码。双方在代码演进中共同更新骨架。

优先级框架

CaTDD 将测试组织为四个优先级级别，并遵循严格的执行顺序。这就是验证的架构。

P0：功能测试（Functional Testing）

公式： $P0 = \text{ValidFunc}(\text{Typical} + \text{Edge}) + \text{InvalidFunc}(\text{Misuse} + \text{Fault})$

P0 是其他一切之前的默认关卡。按照标准顺序，在推进到 P1 之前至少完成 P0 级别的测试——这确保 API 合约的成功路径和失败路径都得到验证。然而，下文所述的针对特定上下文的调整可能会将某些 P1 类别提前，与 P0 的完成交叉进行。

ValidFunc — 证明系统正确工作

类别	目的	示例
Typical ★	核心正常路径 workflow	服务注册、事件发布、命令执行
Edge □	边界值、极限、模式	最小/最大值、空输入、Block/NonBlock/Timeout 模式

InvalidFunc — 证明系统优雅地失败

类别	目的	示例
Misuse ☹️	不正确的 API 使用模式	错误的调用顺序、重复初始化、无效参数
Fault ▲	外部故障与恢复	网络故障、磁盘满、进程崩溃恢复

P1：面向设计的测试（Design-Oriented Testing）

验证架构决策的测试：状态管理、能力与职责边界、协作者交互契约和并发模型。

类别	目的	示例
State 🔄	状态机转换与生命周期	Init→Ready→Running→Stopped
Capability ★	最大容量与系统极限	最大连接数、队列极限、资源池耗尽
Interaction ⇄	协作者顺序与交接契约	编排器→插件调用顺序、先验证后分发、部分失败后回滚
Concurrency ▶	线程安全与竞态条件	并行访问、死锁场景、无锁验证

P2：面向质量的测试（Quality-Oriented Testing）

非功能需求：性能、稳定性、兼容性、配置行为、可观测性和安全性。

类别	目的	示例
Performance »	速度、吞吐量、资源使用	延迟基准测试、内存泄漏检测
Robust ■	压力、重复、长期运行稳定性	1000 次重复、24 小时浸泡测试
Compatibility 🔄	跨平台、版本测试	Windows/Linux/macOS、API 版本兼容
Configuration ≡	设置与部署变体	Debug vs Release、功能开关、环境变量
Diagnosis 🔍	可观测性与失败可解释性	日志中的关联 ID、可操作的错误输出、健康/状态证据
Security ♦	威胁条件下的保护	认证/授权拒绝、机密脱敏、信任边界与注入防御

P3：附加项测试（Addons Testing）

类别	目的
Demo/Example 📌	端到端演示、教程、最佳实践示例

默认测试顺序

```
P0: Typical → Edge → Misuse → Fault
P1: State → Capability → Interaction → Concurrency
P2: Performance → Robust → Compatibility → Configuration → Diagnosis → Security
P3: Demo/Example
```

此顺序并非僵化不变——它会根据上下文进行调整。

针对特定上下文的优先级调整

不同的项目类型需要不同的测试优先级。CaTDD 为常见上下文提供了调整规则：

新的公共 API

```
P0: Typical → Edge → Misuse → Fault（彻底完成 P0）
P1: State → Capability → Interaction → Concurrency
P2: Performance
```

理由：在做高级测试之前，先确保 API 合约的正确性。

有状态/重度 FSM 组件

```
P0: Typical → Edge（基本功能）
P1: State（提前提升）→ Capability → Interaction → Concurrency
P0: Misuse → Fault（完成功能测试）
P2: Performance → Robust
```

理由：状态转换是架构核心——在基本功能之后测试它们。

可靠性关键服务

```
P0: Typical → Edge → Fault（提升）→ Misuse
P1: State → Capability → Interaction → Concurrency
P2: Robust（提升）→ Performance → Compatibility
```

理由：错误处理和稳定性至关重要。

高性能系统（有 SLO）

```
P0: Typical → Edge → Misuse
P2: Performance（在 P2 内提升）→ Robust
P1: State → Capability → Interaction → Concurrency
P0: Fault（完成 P0）
```

理由：性能特征是设计约束，而非事后考虑。

高并发设计

P0: Typical → Edge → Misuse
P1: Concurrency (提升为 P1 第一个) → State → Capability → Interaction
P0: Fault (完成 P0)
P2: Performance → Robust

理由：线程安全是架构基础。

数据处理管道

P0: Typical → Edge → Fault → Misuse
P2: Performance (提升) → Robust (提升)
P1: State → Capability → Interaction → Concurrency

理由：数据完整性和吞吐量是关键质量属性。

风险驱动的优先级调整

当针对特定上下文的调整不够时，使用风险评分公式：

$$\text{Risk Score} = \text{Impact} \times \text{Likelihood} \times \text{Uncertainty}$$

Impact: 1 (低) → 3 (关键)
Likelihood: 1 (罕见) → 3 (频繁)
Uncertainty: 1 (已知) → 3 (未知)

最高分：27

优先级规则：

- 得分 ≥ 18 : 将该类别立即移到 Edge 之后
- 得分 12-17 : 比默认位置提前 2 位
- 得分 9-11 : 比默认位置提前 1 位
- 得分 ≤ 8 : 保持默认位置

评估示例：

多线程队列中的 Concurrency :
Impact: 3 (数据损坏)
Likelihood: 3 (多线程)
Uncertainty: 3 (复杂交互)
得分: 27 → 在 Edge 之后立即测试

批处理器中的 Performance :
Impact: 2 (较慢但仍可用)
Likelihood: 2 (取决于负载)
Uncertainty: 2 (已有部分基准测试)
得分: 8 → 保持默认位置

US/AC/TC 合约

CaTDD 的核心是从人类需求到机器可执行测试的可追溯链路：

User Story (US) → Acceptance Criteria (AC) → Test Case (TC)

↓ ↓ ↓

业务价值 可测试条件 具体断言

用户故事 (User Story) 模板

US-n: 作为 [特定角色/人物画像]，
我想要 [特定的能力或功能]，
以便 [具体的业务价值或收益]。

每个 US 代表一个独特的用户价值。一个模块通常有 2-5 个 User Story。

来自 IOC 事件系统的真实示例：

US-1: 作为高负载场景下的事件生产者，
我希望在队列满时能够不阻塞地发布事件，
以便我的应用在负载下保持响应。

验收标准 (Acceptance Criteria) 模板

AC-n: GIVEN [初始上下文和前置条件]，
WHEN [特定的触发器、动作或事件]，
THEN [预期的可观察结果或行为]，
AND [如有额外的预期结果]。

每个 US 定义 1-4 个 AC。每个 AC 必须可独立验证。

真实示例：

AC-1: GIVEN 一个事件生产者调用 IOC_postEVT_inConclesMode，
WHEN IOC 的 EvtDescQueue 在 ASyncMode 下因阻塞消费者而满，
THEN 生产者立即返回而不等待，
AND 返回 IOC_RESULT_TOO_MANY_QUEUEING_EVTDESC，
AND 该事件未被加入处理队列。

测试用例 (Test Case) 模板

```
[@AC-n, US-n]
TC-n:
  @[Name]: verifyBehavior_byCondition_expectResult
  @[Purpose]: 为什么这个测试重要以及它验证什么
  @[Brief]: 用简单的话描述测试做什么
  @[Steps]: 详细执行步骤 (复杂测试可选)
  @[Expect]: 如何验证成功
  @[Notes]: 额外上下文、注意事项或依赖
```

命名规范：`verifyBehavior_byCondition_expectResult`

```
verifyServiceRegistration_byValidName_expectSuccess
verifyEventPost_byFullQueue_expectNonBlockReturn
verifyCommandExec_byMultipleClients_expectIsolatedExecution
verifyStateTransition_byInvalidSequence_expectError
```

这条链路必须是可追溯的：每个 TC 引用其 AC，每个 AC 引用其 US。CaTDD CodeAgent 读取这条可追溯链路，以理解要生成什么以及为什么。

TDD 红→绿循环 (CaTDD 风格)

CaTDD 通过显式的状态跟踪来规范 TDD 循环：

```
○ TODO/PLANNED → ● RED/FAILING → ● GREEN/PASSED
  (已设计)          (测试已编写，          (测试通过)
                  代码缺失)
```

每个测试用例以 ○ TODO 开始（在注释中设计好）。你实现测试，将其标记为 ● RED（应该失败，因为生产代码尚未编写）。你实现最小生产代码，运行测试，将其标记为 ● GREEN。然后你推进到下一个测试用例。

4 阶段测试结构

每个测试遵循一致的模式：

```
TEST(CategoryName, verifyBehavior_byCondition_expectResult) {
    //===SETUP===
    // 初始化环境、创建资源、配置前置条件

    //===BEHAVIOR===
    printf("▶ BEHAVIOR: verifyBehavior_byCondition_expectResult\n");
    // 执行被测试的动作

    //===VERIFY===
    // 验证结果（保持 ≤3 个关键断言）

    //===CLEANUP===
    // 释放资源、重置状态
}
```

为什么每个测试 ≤3 个断言？

- 更容易识别哪个失败了
- 更好的测试隔离性
- 更清晰的测试目的
- 如果需要更多断言，创建额外的测试用例

质量关卡（Quality Gates）

CaTDD 在各优先级级别之间定义了显式的关卡。在满足所有条件之前，你不能通过关卡。

关卡 P0：离开功能测试之前

必须完成：ValidFunc(Typical + Edge) + InvalidFunc(Misuse + Fault)

- 所有 Typical 测试 GREEN（80–90% 核心 workflows 覆盖）
- 所有 Edge 测试 GREEN（边界情况、边界值、极限已验证）
- 所有 Misuse 测试 GREEN 或已记录（错误用法已处理）
- 所有 Fault 测试 GREEN 或已记录（错误恢复已验证）
- 没有关键的正确性 bug
- Fast-Fail Six 测试全部通过
- 基本的内存/资源泄漏检查通过

退出条件：完整的 API 合约经过测试——成功和失败路径均覆盖。

关卡 P1：进入面向质量的测试之前

- State 测试 GREEN（如果有状态组件）
- Capability 测试 GREEN（极限已表征）

- Interaction 测试 GREEN（如果存在顺序、协作或交接规则）
- Concurrency 测试 GREEN（如果多线程）
- 没有已知的死锁或竞态条件
- ThreadSanitizer/AddressSanitizer 通过
- 架构已验证符合设计要求

关卡 P2：发布之前

- Performance 测试 GREEN（如有 SLO 定义则满足）
- Robust 测试 GREEN（压力/浸泡测试通过）
- Compatibility 测试 GREEN（如果跨平台）
- Configuration 测试 GREEN（如果可配置）
- Diagnosis 测试 GREEN（当需要可观测性或失败可解释性时）
- Security 测试 GREEN（当定义了威胁、策略或保护要求时）
- 满足生产就绪标准

可选关卡 P3：文档完成

- Demo/Example 测试 GREEN
- 教程代码已验证
- 最佳实践已记录

Fast-Fail Six

尽早并经常运行这六个测试，以在详细测试浪费时间之前捕获常见问题：

1. 空/空值输入处理 — 每个 API 是否用正确的错误码拒绝 `NULL` 和空字符串？
2. 零/负数超时 — 超时为 0、-1 或 `UINT_MAX` 时会发生什么？
3. 重复注册/订阅 — 重复注册是否返回 `ALREADY_EXISTS`？
4. 非法调用序列 — 在初始化之前或清理之后调用 API 会发生什么？
5. 缓冲区满/空的边界情况 — 填充到容量上限，再尝试一个。完全清空，再尝试一个。
6. 重复关闭/重新初始化的幂等性 — 系统是幂等的还是返回正确错误？

这六个测试是防范 API 滥用的最低防线。它们在更深入的测试设计开始之前建立合约边界。

测试组织策略

CaTDD 根据项目规模支持两种组织策略：

单文件策略（较简单项目，<50 个测试）

将一个组件的所有测试放在一个文件中。使用 `TEST suite` 按类别组织。适用于小型到中型模块。

多文件策略（较大项目）

按类别划分的测试文件遵循规范命名规则 `test_{feature}_{category}.<ext>`，其中 `{feature}` 是稳定的 `lower_snake_case` 用法切片，`{category}` 是固定的 CaTDD 类别 token：

- `test_{feature}_freelyDrafts.cxx` — 探索与想法捕获（Stage-0 草案）
- `test_{feature}_funcValidTypical.cxx` — 核心 workflow
- `test_{feature}_funcValidEdge.cxx` — 边界情况、边界值、极限
- `test_{feature}_funcInvalidMisuse.cxx` — API 滥用模式
- `test_{feature}_funcInvalidFault.cxx` — 错误处理与恢复
- `test_{feature}_designState.cxx` — 状态转换
- `test_{feature}_designCapability.cxx` — 能力极限与职责
- `test_{feature}_designInteraction.cxx` — 协作者顺序与交接
- `test_{feature}_designConcurrency.cxx` — 线程安全
- `test_{feature}_qualityPerformance.cxx` — SLO 与资源预算
- `test_{feature}_qualityRobust.cxx` — 压力与长期运行稳定性
- `test_{feature}_qualityCompatibility.cxx` — 版本/平台/工具链兼容性
- `test_{feature}_qualityConfiguration.cxx` — 功能开关与环境变化
- `test_{feature}_qualityDiagnosis.cxx` — 可观测性与失败证据
- `test_{feature}_qualitySecurity.cxx` — 威胁下的保护属性
- `test_{feature}_addonDemoExample.cxx` — 教程与文档化示例

成熟、稳定的测试点从 `freely-drafts` 文件迁移到按类别划分的文件；每个 `feature` 都为每个规范 token 保留一个文件（没有适用测试点的类别以 `@[NoTestPoints]: <reason>` 作为活决策保留）。

实现跟踪模板

每个 CaTDD 测试文件包含一个 TODO/实现跟踪部分，记录所有测试用例及其状态：

```
//=====
// ★ 高优先级 - 核心功能
//=====
// ○ [AC-1,US-1] TC-1: verifyCore_byBasicOperation_expectSuccess
// ● [AC-2,US-1] TC-1: verifyCore_byMaxCapacity_expectProperHandling
//
//=====
// ☆ 中优先级 - 边界与错误处理
//=====
// ○ [AC-3,US-1] TC-1: verifyEdge_byEmptyQueue_expectEmptyResult
// ○ [AC-4,US-2] TC-1: verifyMisuse_byDoubleInit_expectError
//
//=====
// ○ 低优先级 - 高级场景
//=====
// ○ [AC-5,US-2] TC-1: verifyPerformance_byHighLoad_expectAcceptableLatency
//
// ▶ GATE P0：所有 P0 测试必须 GREEN 后才能进入 P1。
```

这个跟踪部分是整个测试工作的仪表盘。开发者和 CodeAgent 都通过它了解哪些已完成、哪些进行中、哪些已规划。

设计即活合约

CaTDD 从根本上重新定义了设计。设计不是编码开始之前就完成的、在上游的活动。设计是一份活合约，它存在于测试文件中，随代码一起演进，并且对人类和 LLM 都是可读的。

这份合约声明了：

1. 什么重要 — 以附带业务价值的 User Story 表达
2. 要验证什么 — 以 GIVEN/WHEN/THEN 格式的 Acceptance Criteria 表达
3. 如何验证 — 以带有具体断言的 Test Case 表达
4. 什么优先级 — 以 P0→P1→P2→P3 框架表达
5. 什么状态 — 以 ○→●→● 标记表达

这是 CaTDD 一切的基础。它是 methodPrompts 定义的、slashCommands 操作化的、codeAgents 执行的、agentSkills 打包的。所有其他层都依赖这些概念。

从概念到行动

后续章节从"是什么"转向"怎么做"：

- 第 2 章：chatVibeCoding — 如何在基于聊天的开发中结合 LLM 使用 CaTDD，VibeCoding 与 SpecCoding 的区别
- 第 3 章：callSlashCommands — 如何调用 slash command 系统，进行结构化、可重复的 CaTDD 执行
- 第 4 章：asyncCodeAgent — 代码 agent 如何自动化测试设计、测试实现以及完整的 SpecCoding 生命周期
- 第 5 章：applyClassicSWE — 软件工程知识之书（TDD、BDD、DDD）在 LLM 时代的应用，通过 CaTDD 将三者融为一体

chatVibeCoding

devBooks/ActionZH/02_chatVibeCoding.md

什么是 Vibe Coding ?

Vibe coding 是使用与 LLM 的自然语言对话来生成代码的实践。你描述你想要什么，LLM 产生一个实现，你通过优化你的 prompt 来迭代。

在 CaTDD 中，vibe coding 不是无结构的自由发挥。它是 LLM 交互的第一种模式——对话式、探索性的模式，你和 CodeAgent 一起逐步完成设计。但 CaTDD 增加了一个关键区别：有非结构化的 VibeCoding，也有结构化的 SpecCoding。两者各有其位，方法论会告诉你何时使用每一种。

CaTDD 交互的两种模式

用 CaTDD 的术语来说，在 CodeAgent 聊天中直接使用 `methodPrompts` 就是 VibeCoding：灵活的、方法引导的对话。使用 `slashCommands` 就是 SpecCoding：基于相同方法定义的结构化规约驱动开发流程。

方面	VibeCoding	SpecCoding
定义	灵活的、方法引导的 LLM 对话	带有显式生命周期命令的结构化流程
方法来源	直接使用 <code>methodPrompts</code>	<code>slashCommands</code> （包装 <code>methodPrompts</code> ）
控制	开发者主导对话	流程命令驱动生命周期
工件	注释、测试文件、代码 — 临时性	SpecFlow 工件： <code>pendingNews</code> 、 <code>todoUS</code> 、 <code>doingUS</code> 、 <code>abortUS</code> 、 <code>doneUS</code>
可追溯性	可选，取决于开发者	强制，由流程保证
使用时机	探索、起草、头脑风暴、快速修复	结构化交付、团队协作、可复现的工作流
命令调用	开发者输入自由格式的 prompt	开发者调用 <code>/UT_*</code> 或 <code>/SPEC_*</code> 命令

何时使用 VibeCoding

VibeCoding 是以下场景的正确模式：

1. 你在探索一个新功能 — 你有一个粗略的想法，但需要 LLM 帮你思考覆盖维度、边界情况和风险。你粘贴 API 头文件，问“我应该考虑哪些测试维度？”，然后在 LLM 的建议上迭代。

2. 你在自由起草测试想法 — CaTDD 的 Stage-0（自由起草）就是纯 VibeCoding。你捕获原始的场景、风险、示例和未解决的问题，而不急于做出类别决策。LLM 帮你头脑风暴。
3. 你在澄清需求 — 你粘贴模块 README，让 LLM 识别模糊的验收标准，并讨论正确的行为应该是什么。这是非正式进行的 `SPEC_clearStoryIntent` 阶段。
4. 你在排查问题 — 一个测试失败了，你粘贴错误信息和测试代码，LLM 帮你诊断是 setup 问题、断言问题还是真正的 bug。
5. 你在做一次性工作 — 一个小的验证脚本、配置变更或文档更新。SpecCoding 的开销不值得。

何时切换到 SpecCoding

VibeCoding 有一个自然的边界。当你发现自己处于以下情况时：

- 多次向 LLM 重复相同的指令
- 丢失了已做决策的追踪
- 忘记了哪些测试已设计 vs 已实现
- 需要与团队成员分享进展
- 需要在一天或一周后恢复工作

……就是切换到 SpecCoding 的时候了。VibeCoding 用于探索和对话。SpecCoding 用于结构化交付和团队协作。它们不是竞争模式——它们是同一工作流中互补的阶段。

VibeCoding 如何配合 methodPrompts 工作

在 VibeCoding 模式下，你直接使用 methodPrompts 层。你与 LLM 共享 method prompt，然后以对话方式迭代。

基本模式

1. 共享上下文： 粘贴 `methodPrompts/CaTDD_methodPrompt.md`（或相关类别 prompt）
2. 描述意图： "使用 CaTDD，我为 [组件] 设计测试。"
3. 提供输入： 粘贴 API 头文件、已有测试文件、文档
4. 迭代： 审查、优化、请求澄清、请求替代方案
5. 产出： LLM 生成 US/AC/TC 设计、测试代码或生产代码
6. 保存： 将 LLM 的输出复制到你的测试文件中

示例：VibeCoding 一个 Typical 设计骨架

开发者：

使用 CaTDD 方法论（来自 `methodPrompts/CaTDD_methodPrompt.md`），
帮我为事件队列组件设计 Typical 设计骨架。

关键关注点：

- 生产者异步发布事件
- 消费者拉取事件，支持可选的阻塞模式
- 队列具有固定容量

覆盖维度：Producer × Consumer × Mode
优先级：默认 P0 顺序（Typical → Edge → Misuse → Fault）

CodeAgent（在阅读 method prompt 之后）：

我将为事件队列设计 P0 Functional / Typical 设计骨架。

覆盖矩阵：

Producer	Consumer	Mode	Scenario
Single	Single	Callback	US-1: 基本事件流
Single	Single	Pull	US-2: 轮询流
Multiple	Single	Callback	US-3: 多对一汇聚

用户故事 (User Stories)：

US-1: 作为事件生产者，
我希望将事件发布到队列以进行异步处理，
以便我的生产者线程不被消费者工作阻塞。

US-2: 作为事件消费者，
我希望从队列中拉取事件，支持可选的阻塞模式，
以便我可以在准备就绪时处理事件，无需忙等待。

US-3: 作为系统操作员，
我希望多个生产者安全地向同一队列发布事件，
以便来自不同来源的事件被正确处置。

...

对话继续：开发者审查，请求边界情况覆盖，要求 CodeAgent 生成实现，并迭代优化。这就是 VibeCoding——LLM 是应用 CaTDD 方法的对话伙伴。

VibeCoding workflow

CaTDD 的 VibeCoding 遵循方法各阶段的自然推进：

阶段 1：理解（方法引导的探索）

你对 LLM 说的话：

"我在构建一个 [组件]。这是 API 头文件。
使用 CaTDD，我应该测试什么以及如何排优先级？"

LLM 做什么：

- 读取你的接口
- 检查已有的相关测试
- 识别依赖和约束
- 呈现理解摘要
- 在继续之前提出澄清问题

CaTDD 合约：LLM 应遵循 methodPrompt 中的 Agent Workflow Checklist（阶段 1：理解）。它在设计任何东西之前，应读取文件、识别 API、找到类似测试并澄清模糊之处。

阶段 2：设计（注释优先，尚未编码）

你说的话：

"好的。现在设计覆盖矩阵和 User Stories。"

LLM 做什么：

- 填充 OVERVIEW 部分 (WHAT/WHERE/WHY/SCOPE)
- 定义覆盖矩阵维度
- 编写 User Stories (2-5 个, As a/I want/So that 格式)
- 编写 Acceptance Criteria (GIVEN/WHEN/THEN)
- 使用结构化元数据详细说明 Test Cases
- 填充 TODO 跟踪部分

CaTDD 合约：先设计后编码。LLM 在生成任何测试实现之前，应首先编写注释——US/AC/TC 链路。这就是“评论即设计”的部分：设计以注释形式存在，并与代码共存。

阶段 3：实现 (TDD 红→绿)

你说的话：

"实现第一个 Typical 测试用例。遵循 RED→GREEN。"

LLM 做什么：

- 编写测试实现 (4 阶段结构)
- 在 TODO 部分将其标记为 ● RED
- 确认它失败 (应该失败，因为代码缺失)
- 实现最小生产代码
- 确认它通过
- 将其标记为 ● GREEN

CaTDD 合约：LLM 应遵循严格的 TDD 纪律。绝不应在编写失败测试之前实现生产代码。每次动作后应立即更新状态标记。

阶段 4：迭代与优化

你说的话：

"现在做 Edge 测试。并添加队列满行为的测试。"

LLM 做什么：

- 按照优先级框架继续推进
- 逐个 TC 推进 RED→GREEN 循环
- 在每个质量关卡处停下来进行审查
- 报告覆盖缺口并推荐下一步

VibeCoding vs SpecCoding：开发者的决策

VibeCoding 和 SpecCoding 之间的选择是一个光谱，而非二元开关。你可以从 VibeCoding 开始，到达自然的转折点，然后切换到 SpecCoding。

从 VibeCoding 开始的情况

- 组件是新的，需求正在浮现
- 你是唯一的开发者
- 你需要在承诺结构化之前探索
- 范围较小（<5 个 User Stories，<20 个测试用例）
- 你想要快速迭代，不需要生命周期开销

切换到 SpecCoding 的情况

- 需求已经稳定，需要可追溯性
- 你正在与团队成员协作
- 你需要跨天暂停和恢复工作
- 范围较大（5+ 个 User Stories，20+ 个测试用例）
- 你想要可复现的 `/UT_*` 和 `/SPEC_*` 命令序列
- 利益相关者需要了解进展（pendingNews → todoUS → doingUS → doneUS，或当活跃故事不宜继续时使用 abortUS）

自然过渡点

最常见的过渡点是在 Stage-0 自由起草之后。你在 VibeCoding 模式中探索、头脑风暴、讨论风险、草绘测试场景。当你对草案有足够清晰的认识，可以将其分类到 CaTDD 类别中时（Stage-1），你切换到 SpecCoding 并开始调用结构化命令。

VibeCoding: 探索 → 头脑风暴 → 起草 → 讨论
 ↓ (转折点)
SpecCoding: 分类 → 设计骨架 → 审查 → 实现 → 提交 → 关闭

Method Prompt 作为聊天上下文

在 VibeCoding 中，method prompt 是你与 LLM 的共享上下文。你不需要每次都重新解释 CaTDD——你共享一次 method prompt，LLM 在整个对话中都会应用它。

需要共享什么

有效 VibeCoding 的最小上下文：

1. 主 method prompt（`CaTDD_methodPrompt.md`）—— 建立方法论合约：优先级框架、类别定义、US/AC/TC 格式、状态跟踪、质量关卡。
2. 相关类别 prompt（`CaTDD_methodPrompt4Cat-Typical.md` 等）—— 在特定类别下工作时，共享该类别的专门指导。
3. API 接口—— 被测组件的头文件、规约文档或协议定义。
4. 已有测试文件—— 以便 LLM 遵循已有模式并避免重复。

多少上下文足够？

CaTDD method prompt 是全面的（1500+ 行规约说明）。你不需要将它全部粘贴到每次对话中。而是：

- 首次对话：共享完整的 method prompt。LLM 现在理解 CaTDD 了。
- 后续对话：引用它："继续像之前一样使用 CaTDD 方法。"
- 聚焦的对话：仅共享特定工作相关的类别 prompt。

LLM 的上下文窗口是你的约束。优先级如下：

1. 组件接口和规约文档（必须——LLM 需要知道测试什么）
2. 相关类别 prompt（重要——LLM 需要知道如何测试）
3. 已有测试代码（有帮助——LLM 遵循惯例）

VibeCoding 的模型层级策略

VibeCoding 让你与 LLM 直接对话。你选择的模型会影响决策质量。CaTDD 建议使用能保持决策质量的最小模型层级：

层级	在 VibeCoding 中何时使用
SOTA 推理（GPT-5.5-xHh）	架构决策、系统边界、质量权衡、跨模块约束。当对话涉及不可逆的设计选择时使用。
高性能	需求分析、US/AC/TC 设计、测试设计、代码审查。当推理需要跨越多个工件（规约、测试、生产代码）时使用。
闪速	确定性任务：编写单个测试实现、更新 TODO 状态、简单重构。当输入清晰且动作范围狭窄时使用。

升级规则：如果闪速模型产出的设计看起来肤浅，升级到高性能。如果高性能模型做出的架构决策感觉有风险，升级到 SOTA。

常见的 VibeCoding 模式

模式 1：设计审查循环

```
开发者："为这个 API 设计 Typical 设计骨架。"
CodeAgent：[产出 US/AC/TC 设计]
开发者："US-2 没有体现非阻塞需求。修改一下。"
CodeAgent：[修改 US-2 并更新受影响的 AC 和 TC]
开发者："好。现在添加超时模式的覆盖。"
CodeAgent：[为超时行为添加 AC-4 和 TC]
开发者："看起来完整了。切换到 SpecCoding 开始实现。"
```

这是最自然的模式：开发者审查 LLM 的设计输出并指导优化。LLM 保持可追溯性（更新一个 US 会级联影响关联的 AC 和 TC）。

模式 2：覆盖缺口分析

```
开发者："这是我当前的测试文件。你看到哪些覆盖缺口？"  
CodeAgent：[对照优先级框架分析已有的 US/AC/TC 链路]  
CodeAgent："缺失：P0 Edge（队列满行为）、P0 Misuse（重复关闭）、  
P1 State（清理后的生命周期转换）。"  
开发者："先设计 Edge 设计骨架。然后我们做 Misuse。"
```

LLM 使用 CaTDD 优先级框架作为对照已有测试的检查清单。它识别应该存在但缺失的内容。

模式 3：排查会话

```
开发者："TC-3 失败了。期望 IOC_RESULT_INVALID_PARAM，实际得到 success。"  
CodeAgent：[读取测试代码、读取生产代码]  
CodeAgent："测试调用了 IOC_post(NULL, ...)，但生产代码有一个  
guard clause 为 NULL 句柄返回 SUCCESS。这可能是：  
A) 生产代码 bug（应该返回 INVALID_PARAM），或  
B) 测试误解（此 API 允许 NULL）。  
哪个是该 API 合约的正确行为？"
```

LLM 充当诊断伙伴，将测试期望与生产行为进行比较，并揭示合约模糊性。

模式 4："下一步做什么"查询

```
开发者："我已经完成了所有 P0 Typical 测试。下一步应该做什么？"  
CodeAgent：[读取 TODO 跟踪部分]  
CodeAgent："你有 4 个 Edge 测试用例在 ○ TODO、2 个 Misuse 用例在 ○ TODO、  
1 个 Fault 用例在 ○ TODO。按照 P0 默认顺序，  
接下来实现 Edge 用例。TC-Edge-1 依赖于你尚未实现的 capacity API—  
考虑将其标记为 BLOCKED，改为推进 TC-Edge-2。"
```

LLM 读取 CaTDD 跟踪部分，根据优先级规则和依赖分析推荐下一步行动。

VibeCoding 中的两个设计阶段

CaTDD method prompt 支持两个设计阶段，它们自然地映射到 VibeCoding：

Stage-0：自由起草

这是纯 VibeCoding。你捕获原始场景、风险、示例和未解决的问题，而不强制做出类别决策。目标是覆盖广度，而非结构。

你说的话：

```
"我在构建一个事件队列。什么可能出问题？我应该测试什么？"
```

LLM 产出的内容：

FreelyDrafts:

- 如果队列满了，生产者尝试发布会怎样？
- 如果消费者尝试从空队列拉取会怎样？
- 如果多个生产者同时发布会怎样？
- 如果消费者阻塞等待永远不会到达的事件会怎样？
- 如果队列容量为 0—应该允许吗？
- 如果我们在队列被销毁后发布会怎样？
- 如果事件大小差异巨大会怎样？
- 如果回调消费者抛出异常会怎样？
- 风险：如果回调尝试向同一队列回发事件，消费者死锁
- 风险：如果事件发布后从未被消费，内存泄漏
- 问题：零容量队列应该是有效配置吗？
- 问题：我们应该支持基于优先级的事件排序吗？

Stage-0 的输出故意是非结构化的。目标是捕获想法。结构在 Stage-1 中到来。

Stage-1：分类设计

在起草之后，你在编写最终的 US/AC/TC 之前将草案分类到 CaTDD 类别中。这是 VibeCoding 和 SpecCoding 之间的桥梁。

你说的话：

"将这些草案分类到 CaTDD 类别中，并设计骨架。"

LLM 产出的内容（按类别结构化）：

```
P0 Functional / Typical :
  US-1: 作为生产者，我希望发布事件以便它们排队等待处理。
  AC-1: GIVEN 有可用容量的队列，WHEN 生产者发布，THEN 事件被加入队列。
  TC-1: verifyEventPost_byAvailableCapacity_expectEventQueued

P0 Functional / Edge :
  US-2: 作为生产者，我希望队列满时具有非阻塞行为。
  AC-2: GIVEN 队列满，WHEN 生产者以 NonBlock 模式发布，THEN 立即返回并带有错误。
  TC-2: verifyNonBlockPost_byFullQueue_expectImmediateReturn

P0 Functional / Misuse :
  US-3: 作为开发者，我希望对于不正确的队列使用有正确的错误码。
  AC-3: GIVEN 已销毁的队列，WHEN 生产者发布，THEN 返回 INVALID_STATE。
  TC-3: verifyPost_byDestroyedQueue_expectInvalidState

P0 Functional / Fault :
  US-4: 作为系统操作员，我希望消费者故障后能优雅恢复。
  AC-4: GIVEN 消费者回调崩溃，WHEN 已发布的事件保留，THEN 队列保持可操作。
  TC-4: verifyQueue_byConsumerCrash_expectQueueRemainsOperational
```

Stage-1 弥合了自由探索和结构化执行之间的鸿沟。一旦分类完成，你就可以切换到 SpecCoding 并调用 slash command 来实现每个骨架。

带有实时反馈的 VibeCoding

VibeCoding 的优势之一是即时反馈。你立即看到 LLM 的输出并进行迭代。这创建了一个紧密的反馈循环：

```
你的 prompt → LLM 响应 → 你的审查 → 优化后的 prompt → 更好的响应
      |                                     |
      |——— 持续优化 ———|
```

这个循环很快但很脆弱。如果你不保存输出（将它们复制到测试文件中、更新 TODO 状态），对话就成了决策的唯一记录。当聊天过期时，设计就丢失了。

VibeCoding 保存规则：在每次重要的设计输出后，将其复制到文件中。在每次测试实现后，运行它并更新 TODO 部分。不要让聊天成为你唯一的真实来源。

CodeAgent 在 VibeCoding 中的角色

在 VibeCoding 中，CodeAgent 是一个理解方法的对话伙伴。它：

- 读取 method prompt 以理解 CaTDD 语义，而非仅仅是泛化的 TDD
- 保留注释骨架合约 —— 它知道 @[US]、@[AC]、@[TC] 标记是结构性的，而非装饰性的
- 遵循优先级纪律 —— 它在 P1 之前设计 P0，在 Edge 之前设计 Typical
- 跟踪状态 —— 随着对话的推进更新 ○→●→● 标记
- 提问而非假设 —— 当产品意图不明确时，它提问而不是猜测
- 报告它不知道的内容 —— 它呈现缺失的信息，而非用听起来自信的输出来掩盖缺口

CodeAgent 不取代开发者的判断。它通过应用方法一致性和生成结构化输出的速度——比人类打字更快——来放大开发者的判断力。

VibeCoding 反模式

反模式 1：永远 VibeCoding

有些开发者无限期地停留在 VibeCoding 模式中，把每次 LLM 对话都当作临时性的。没有结构，你会积累：

- 孤立的测试用例，没有到需求的可追溯性
- 通过但不验证任何有意义内容的测试（“绿但空洞”问题）
- 跨文件不一致的命名、结构和优先级顺序
- 仅存在于过期聊天会话中的、丢失的设计决策

解决办法：当设计稳定时切换到 SpecCoding。从对话迁移到结构化工件。

反模式 2：在 VibeCoding 中跳过设计

VibeCoding 的速度可能诱使你直接跳到实现：“帮我为事件队列写一个测试。”LLM 生成测试代码，你运行它，它通过。感觉很高效。但是：

- 没有 US/AC/TC 可追溯性：这个测试验证了什么业务价值？
- 没有优先级分类：这是 Typical、Edge 还是一个随机测试？
- 没有覆盖推理：什么应该被测试但尚未被测试？

解决办法：即使在 VibeCoding 模式下，也要遵循“先设计后编码”的原则。要求 LLM 先设计骨架，审查它，然后再实现。

反模式 3：上下文饥饿

你没有提供 API 头文件、已有测试模式或项目惯例，就要求 LLM 设计测试。LLM 用看似合理但错误的假设填补缺口。

解决办法：在要求设计之前，提供：

- Method prompt（以便 LLM 遵循 CaTDD）
- 组件接口（以便 LLM 知道测试什么）
- 已有测试示例（以便 LLM 遵循惯例）
- 你的具体关注点或风险领域（以便 LLM 正确排优先级）

反模式 4：静默批准

LLM 产出了一个设计。你读了，它看起来合理，你说"实现"。但"看起来合理"不等于"正确"。

解决办法：对照 CaTDD 合约明确审查每个输出：

- User Stories 是否表达了真正的用户价值？
- Acceptance Criteria 是否精确地使用了 GIVEN/WHEN/THEN？
- Test Cases 是否追溯回 AC 和 US？
- 类别是否正确（这真的是 Edge 而不是 Misuse？）？
- 断言测试的内容是否正确？

从 VibeCoding 到 SpecCoding：桥梁

VibeCoding（第 2 章）和 SpecCoding（第 3 章）之间的桥梁就是 slash command 系统。当你准备好从对话迁移到结构化执行时：

1. 保存 VibeCoding 输出：将 US/AC/TC 设计作为注释骨架复制到测试文件中。
2. 初始化 SpecFlow：运行 `/SPEC_initProjectContext` 建立共享项目上下文。
3. 导入工作：使用 `/SPEC_importFeature` 或 `/SPEC_importIssue` 创建可追溯的故事工件。
4. 打开故事：使用 `/SPEC_openUserStory` 将工作移入活跃状态。
5. 开始结构化执行：使用 `/UT_designTypicalSkeleton`、`/UT_implTestCase` 及其他 slash command，用可复现、可追溯的步骤按优先级框架推进工作。

VibeCoding 发现了设计。SpecCoding 交付它。两者共同构成了完整的 CaTDD 工作流。

callSlashCommands

devBooks/ActionZH/03_callSlashCommands.md

什么是 Slash 命令？

Slash 命令是 CaTDD 的操作层——它们将 methodPrompts 转化为可执行的命令流程，供任何 CodeAgent 消费使用。Slash 命令不是传统意义上的 CLI 命令。它是一个 Markdown 提示词文件，精确定义了 CodeAgent 该做什么：当前执行哪一步、读取什么、生成什么、保留什么，以及下一步应该执行什么命令。

这一层位于 slashCommands/ 目录中，严格遵循一条架构原则：slashCommands 不重新定义 CaTDD 的方法语义。它将 methodPrompts 中所定义的内容转换为可移植、可重复的命令单元。当 slash 命令与 methodPrompts 发生冲突时，以 methodPrompts 为准。

三大命令族系

Slash 命令按不同用途分为三个族系：

UT 命令 —— 单元测试设计与实现

```
UT_<verb><Object>
```

UT 命令执行 CaTDD 的测试开发步骤。它们操作的对象是测试文件、设计骨架和实现工作制品。它们属于特定类别的流程（P0、P1、P2）。

示例：

- UT_designTypicalSkeleton —— 设计 Typical 类别设计骨架
- UT_implTestCase —— 按 RED→GREEN 循环实现单个测试用例
- UT_reviewFuncTestsSkeleton —— 在实现前审查完整的 P0 功能骨架集
- UT_tellMeNextImplTest —— 从 TODO 跟踪区选择下一个要实现的测试用例

SPEC 命令 —— SpecCoding 生命周期编排

```
SPEC_<verb><Object>
```

SPEC 命令驱动工作沿着可追溯的 SpecCoding 生命周期前进。它们操作的是生命周期工作制品——项目上下文、待处理消息、用户故事、设计文档、测试、产品代码和提交/CI 状态——而非直接操作测试类别或类别骨架。

示例：

- SPEC_initProjectContext —— 创建首个项目上下文文件

- `SPEC_analyzeIssue` —— 将 Issue 转化为可追溯的用户故事
- `SPEC_takeArchDesign` —— 产出高层架构设计
- `SPEC_designUnitTests` —— 进入 CaTDD 测试设计，通常经 P0/P1/P2 流程
- `SPEC_commitWorks` —— 准备并提交已完成的工作
- `SPEC_closeUserStory` —— 将审核通过的已完成工作移至 done 归档

HARNESS 命令 —— Harness 工程工具点

```
HARNESS_<verb><Object>
```

HARNESS 命令是运维性的工具点命令，而非用户故事生命周期命令。它们围绕方法源维护 CaTDD harness：已安装目标项目资产、原生适配器包装、执行与诊断，以及向原始 CaTDD 仓库的受控补丁回写。它们绝不移动 SpecFlow 生命周期状态，也绝不重新定义 CaTDD 类别含义。

示例：

- `HARNESS_verifyInstallation` —— 验证已安装的 CaTDD 目标项目包含完整、一致的适配器资产
- `HARNESS_diagnoseInstallation` —— 诊断安装失败或异常的 CaTDD 安装并推荐修复
- `HARNESS_diagnoseProject` —— 在不改动源码的前提下诊断仓库正确性与 SpecCoding/CaTDD 漂移
- `HARNESS_newTaskSession` —— 捕获会话上下文，使新的 CodeAgent 会话无需重新调查即可续接
- `HARNESS_evolveHarness` —— 从已验证的成功/失败中学习，并演进最窄的规范属主
- `HARNESS_patchCaTDDSource` —— 将有效的已安装项目 CaTDD 改进补丁回写原始源仓库

通用命令模板

每个 slash 命令都遵循 `UT_slashCommandTemplate.md` 和 `SPEC_slashCommandTemplate.md` 中定义的同 Markdown 结构：

区域	内容
Command Header	命令名、流程、CaTDD 类、类别、方法来源、适配器目标
CoT Pattern	声明的推理模式——ReACT、ToT 或 Linear——附该模式的执行步骤
WHO	谁调用此命令、谁应执行此命令
WHAT	此命令具体做什么——它执行的单个工作流步骤
WHEN	有效的启动条件、何时不应使用、前序/后续命令
WHERE	输入文件、输出文件、方法引用、相关流程文档
WHY	开发者价值、CaTDD 方法理由、如何减少歧义
HOW	执行流程——读取什么、保留什么、执行什么、报告什么
Subagent Recommendation	（仅 SPEC 命令）此命令是否适合后台子代理委派
Input Contract	使用可移植占位符的命令参数
Output Contract	期望的响应格式——摘要、涉及的文件、下一步命令
CodeAgent Compatibility	纯 Markdown，无工具特定假设

这种一致性意味着每个 CodeAgent (Copilot、Cline、Continue、utCodeAgentCLI) 都能使用相同的结构来解析和执行任何 slash 命令。

四种命令流程与 HarnessKits 套件

命令按四种流程加一个运维套件组织：

```
slashCommands/flows/
├── P0-FuncTestsFlow.md    ← UT 命令：功能测试设计
├── P1-DesignTestsFlow.md ← UT 命令：设计导向测试
├── P2-QualityTestsFlow.md ← UT 命令：质量导向测试
└── Px-SpecFlow.md        ← SPEC 命令：完整生命周期编排

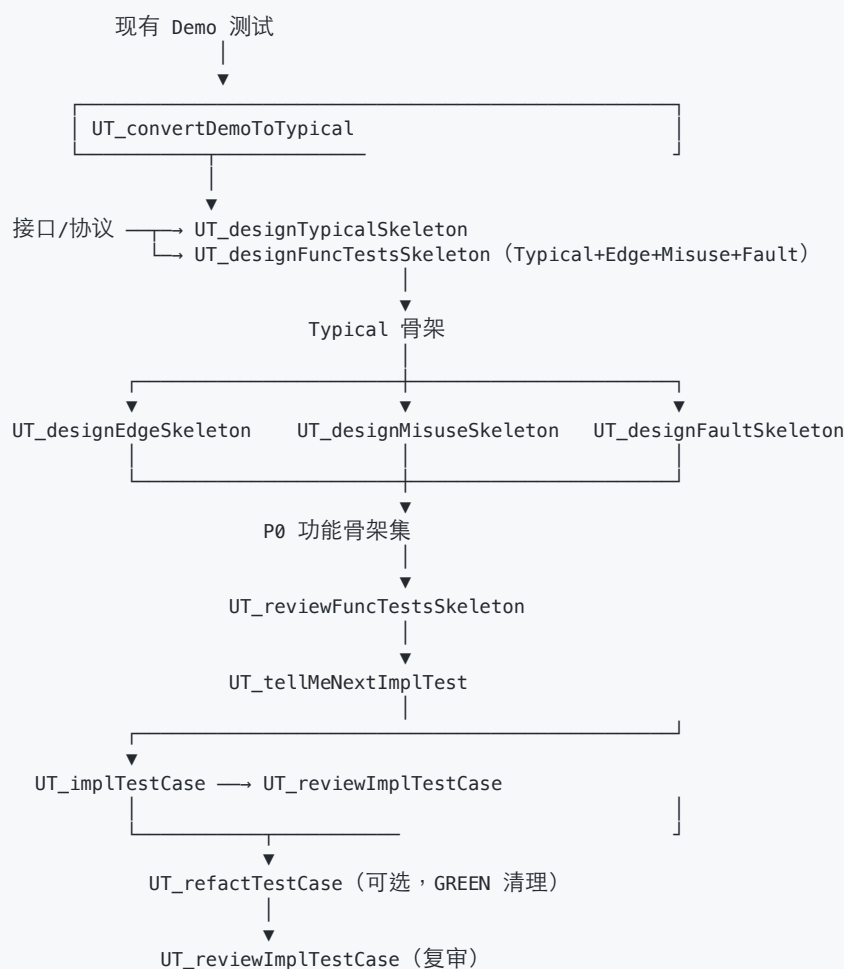
slashCommands/kits/
└── Px-HarnessKits.md     ← HARNESS_* 运维工具点
```

SpecFlow 中的 Px 意为"跨优先级"——它并非像 P0/P1/P2 那样的测试类别优先级，而是一个编排各测试层的流程。

Px-HarnessKits 同样是跨优先级的，但它是一个套件而非生命周期流程：它分组工具点命令，不要求严格的命令顺序。未来的附加/演示命令应属于 P3 Addons，以与 methodPrompts 保持一致。

P0 FuncTestsFlow —— 第一个需要学习的流程

P0 是最常见的入口点，因为它涵盖了功能测试设计——每个开发者首先需要的内容。流程图中的命令序列如下：



入口点

你可以从两个位置启动 P0 流程：

1. 现有 Demo 测试：如果你有用以演练组件的 Demo/Example 代码，使用 `UT_convertDemoToTypical` 将核心行为提炼为 CaTDD Typical 骨架。这能保留已有的工作成果，而非从零开始。
2. 接口或协议：如果你有 API 头文件或规格文档，使用 `UT_designTypicalSkeleton` 从接口契约出发设计 Typical 骨架。
3. 一次性完成完整功能集：当你想将完整 P0 集作为单一行为设计时，使用 `UT_designFuncTestsSkeleton` 一次性产出全部四个骨架（Typical + Edge + Misuse + Fault）。

逐命令详解

选项 A：UT_convertDemoToTypical

何时使用：当你已有演示组件用法的 Demo 测试时。这些并非 CaTDD P3 Demo/Example 测试——它们是捕获核心工作流的原始输入素材。

做什么：读取 Demo 测试，提取核心行为模式，将其映射为 CaTDD Typical 类别格式，并写出带有 US/AC/TC 结构的 Typical 设计骨架。

输出：目标测试文件中的 CaTDD Typical 骨架，其中填充了 `@[US]`、`@[AC]`、`@[TC]` 标记和 OVERVIEW 区域。

下一步命令：`UT_designEdgeSkeleton`

选项 B：`UT_designTypicalSkeleton`

何时使用：从接口或协议出发——没有现成的 Demo 代码时。

做什么：读取接口规格，分析 API 契约，识别核心正向路径场景，并以用户故事、验收标准和测试用例设计 Typical 骨架。

输出：具有完整 US/AC/TC 链的 Typical 设计骨架。

下一步命令：`UT_designEdgeSkeleton`

步骤 3-5：`UT_designEdgeSkeleton`、`UT_designMisuseSkeleton`、`UT_designFaultSkeleton`

每个命令通过读取现有骨架（以保持一致性）、API 接口和对应类别的 method prompt，设计一个类别骨架。它们遵循 P0 顺序：Edge 在 Typical 之后，Misuse 在 Edge 之后，Fault 在 Misuse 之后。

替代方案：当你想一次性设计完整的功能测试集时，可使用 `UT_designFuncTestsSkeleton` 将全部四个 P0 骨架（Typical + Edge + Misuse + Fault）作为单一行为来设计。

重要：每个骨架在不同类别中引用相同的 US ID。例如，US-1 覆盖 Typical 场景，而同一个 US-1 也可能有 Misuse 场景（该能力在同一个用户故事中被错误使用时会怎样）。

步骤 6：`UT_reviewFuncTestsSkeleton`

何时使用：所有 P0 骨架已设计但尚未实现时。

做什么：审查完整的 P0 功能骨架集，检查以下方面：

- 完备性：四种类别是否全部具备？
- 可追溯性：每个 TC 是否引用了 AC，每个 AC 是否引用了 US？
- 类别正确性：Misuse 场景是否被正确归类（没有误入 Edge）？
- 一致性：骨架命名、状态标记和格式是否遵循约定？
- 关卡就绪度：设计是否可进入 P0 实现阶段？

输出：包含每个质量维度 PASS/FAIL 以及具体待修复问题的审查报告。

下一步命令：若 PASS → `UT_tellMeNextImplTest`。若 FAIL → 回到对应的设计命令。

重要：永远不要跳过这个审查关卡。设计阶段发现的缺陷，其修复成本仅为后期实现阶段发现缺陷的十分之一。

步骤 7：`UT_tellMeNextImplTest`

何时使用：准备开始实现——所有骨架已审查通过。

做什么：读取所有测试文件中的 TODO 跟踪区，评估 P0 默认顺序（Typical → Edge → Misuse → Fault），检查依赖关系（例如 TC-3 是否因能力 API 尚不存在而被阻塞？），并推荐下一个要实现的测试用例。

输出：下一个 TC 的名称、所在文件、类别、预估复杂度和所有依赖关系或阻塞因素。

步骤 8：UT_implTestCase

何时使用：已选定一个具体的 TC 准备实现时。

做什么：遵循严格的 TDD 循环实现一个测试用例：

1. 编写测试实现（四阶段：SETUP/BEHAVIOR/VERIFY/CLEANUP）
2. 在 TODO 区域将其标记为 ● RED
3. 运行测试——确认 RED（因缺少产品代码而失败）
4. 实现最小量产品代码
5. 运行测试——确认 GREEN（通过）
6. 在 TODO 区域将其标记为 ● GREEN
7. 使用实现说明更新 TC 规格

输出：一个已实现的测试用例，包含 RED→GREEN 历史记录和更新后的状态标记。

下一步命令：UT_reviewImplTestCase

步骤 9：UT_reviewImplTestCase

做什么：审查以下实现方面：

- 正确性：测试是否确实验证了它声称要验证的 AC？
- 结构：是否遵循四阶段模式？
- 断言：是否有 ≤3 个关键断言且具有实质意义？
- 可追溯性：US/AC/TC 链是否得到保留？
- 隔离性：清理是否执行并防止状态泄漏？
- 状态：标记是否正确设置？

输出：包含 PASS/FAIL 的审查报告。若 PASS，循环回到 UT_tellMeNextImplTest 获取下一个 TC。若 FAIL，返回修复。

步骤 10：UT_refactTestCase（可选）

何时使用：选定的 TC 已 GREEN 且已审查，但测试体难以阅读或维护。

做什么：在不改变行为、类别路由或 US/AC/TC 可追溯性的前提下，重构该单个测试用例以提高清晰度。缺失行为、缺失覆盖或错误类别路由必须回到设计/实现命令，而非在重构中加入。

下一步命令：再次执行 UT_reviewImplTestCase——证明清理后没有骨架漂移。

Slash 命令中的 RED→GREEN 纪律

UT_implTestCase 命令强制执行严格的 TDD 纪律。详细过程如下：

RED 阶段

1. 读取 TC 规格（评论即设计）
2. 按四阶段模式编写测试函数
3. 在测试函数上方添加 @[Name] 和 @[Steps] 注释
4. 更新 TODO 跟踪区：○ → ● RED/FAILING
5. 编译并运行—验证测试失败
6. 如果测试通过：进行调查，可能是以下原因之一：
 - a) 测试编写不当（未测试正确的内容）
 - b) 功能已存在（先写了产品代码—违反 TDD！）
 - c) 测试框架配置有误
7. 报告："RED 确认。测试按预期失败，因为 [原因]。准备实现产品代码。"

GREEN 阶段

1. 阅读测试代码，理解需要哪些产品代码
2. 编写最小量产品代码—仅足以让此测试通过
3. 不要实现未经测试的功能，也不要过度设计
4. 编译并运行—验证测试通过
5. 更新 TODO 跟踪区：● → ● GREEN/PASSED
6. 运行整个测试文件—验证无回归
7. 提交变更
8. 报告："GREEN 确认。测试通过。P0 中剩余 [N] 个测试。"

"最小量产品代码"规则

这是关键原则。当 `UT_implTestCase` 处于 GREEN 阶段时，它必须实现仅足以让当前测试通过的产品代码。一行都不多。这防止了：

- 死代码：编写了但从未被测试的代码
- 过度设计：实现了没有任何测试要求的功能
- 虚假信心：测试因额外代码意外满足而通过
- 可追溯性丢失：产品代码缺少对应的 TC

P1-DesignTestsFlow 和 P2-QualityTestsFlow

P1 和 P2 流程遵循与 P0 相同的结构，但针对各自的类别。然而，它们有一个 P0 不具备的关键约束：设计源关卡（Design Source Gate）。

设计源关卡

P1 和 P2 命令在开始起草任何骨架之前，需要有已确认的设计源。这防止了在架构决策缺失的情况下设计测试：

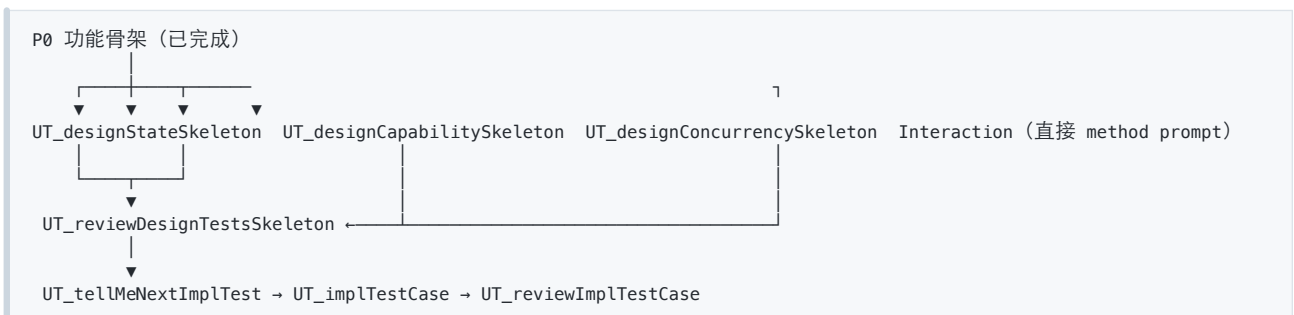
优先级	类别	必需的设计源
P1	State	README_StateDesign.md 或 README_ArchDesign.md 中的 State Design 章节
P1	Capability	README_DetailDesign.md
P1	Interaction	README_ArchDesign.md 中的顺序、交互或协作章节
P1	Concurrency	README_ResourceDesign.md
P2	Performance	README_PerfDesign.md

优先级	类别	必需的设计源
P2	Robust	README_ErrorDesign.md
P2	Compatibility	README_CompatDesign.md
P2	Configuration	README_DetailDesign.md
P2	Diagnosis	README_DiagnosisDesign.md / README_VerifyDesign.md 证据要求
P2	Security	SecurityDesign 文档、威胁模型或安全策略

如果缺少必需的设计源，命令会在起草骨架前发出警告并停止，询问开发者设计在哪里，而不是凭空编造架构决策。这个关卡贯彻了一个原则：P1 和 P2 测试验证的是架构决策，而不仅是 API 行为。

P1-DesignTestsFlow

处理架构验证：State、Capability、Interaction 和 Concurrency。



入口条件：P0 功能骨架必须存在（尤其是 Typical 和 Edge）。每个 P1 类别都需要已确认的设计源。P1 在骨架起草开始之前必须有 DESIGN。

路由说明：State、Capability 和 Concurrency 有专门的 `UT_design*Skeleton` 命令。Interaction 目前还没有专门的 slash 命令——流程直接通过 `CaTDD_methodPrompt4Cat-Interaction.md` 路由，在相同的 Discovery Gate 与审查契约下，将源关联的 US/AC/TC 写入规范的 `designInteraction` 类别文件。

关卡 P1：进入 P1 前，所有 P0 测试必须为 GREEN。离开 P1 前，架构必须经过验证：存在状态行为时 State 测试 GREEN，存在设计极限时 Capability 测试 GREEN，存在顺序/协作/交接规则时 Interaction 测试 GREEN，存在共享执行时 Concurrency 测试 GREEN——且无死锁、无竞态条件、ThreadSanitizer 干净。

P2-QualityTestsFlow

处理非功能质量属性：Performance、Robust、Compatibility、Configuration、Diagnosis、Security。



入口条件：P0 功能覆盖存在，相关的 P1 设计覆盖存在，且每个 P2 类别都有已确认的项目根设计源。

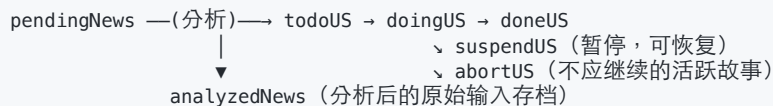
路由说明：Performance、Robust、Compatibility 和 Configuration 有专门的 `UT_design*Skeleton` 命令。Diagnosis 和 Security 目前还没有专门的 slash 命令——流程直接通过 `CaTDD_methodPrompt4Cat-Diagnosis.md` 和 `CaTDD_methodPrompt4Cat-Security.md` 路由，将源关联的 US/AC/TC 写入规范的 `qualityDiagnosis` 与 `qualitySecurity` 文件。

关卡 P2：进入 P2 前，P1 必须完成。离开 P2 前，质量 SLO 必须满足，所需诊断/可观测性证据已验证，存在威胁模型或策略时安全保护测试 GREEN，生产就绪标准必须达标。

Px-SpecFlow —— 主编排流程

Px-SpecFlow 是最大、最重要的流程，因为它编排完整的 SpecCoding 生命周期。UT 流程执行测试相关的具体步骤，而 SPEC 命令则管理从导入到关闭整个故事生命周期。

SpecFlow 生命周期



每个工作项都会经历已分析、活跃与归档通道，其中 `abortUS` 和 `suspendUS` 用于保留不应继续进行的活跃故事：

阶段	目录	含义
pendingNews	<code>.catdd/spec/pendingNews/</code>	等待分析的工作。原始 Issue 与功能请求。
analyzedNews	<code>.catdd/spec/analyzedNews/</code>	分析后存档的原始输入——保留源追溯。
todoUS	<code>.catdd/spec/todoUS/</code>	已分析、可领取的工作。结构化用户故事及候选验收标准。
doingUS	<code>.catdd/spec/doingUS/</code>	正在进行中的活跃工作。处于设计、测试或实现阶段的已开启用户故事。
suspendUS	<code>.catdd/spec/suspendUS/</code>	暂停的活跃故事，附持久恢复引用（分支/worktree），供暂停而非就地继续时使用。
abortUS	<code>.catdd/spec/abortUS/</code>	已中止的活跃工作，保留以供后续分析或作为下一轮改进的输入。
doneUS	<code>.catdd/spec/doneUS/</code>	已完成的工作。经审查、已提交、CI 通过的故事。

完整命令序列

完整的 SpecFlow 生命周期包含 34 个 SPEC 命令，分为三个阶段：

阶段 C：实现与关闭

若为实现导向

17. SPEC_designUnitTests
经 P0/P1/P2 流程进入 CaTDD 测试设计
 18. SPEC_implUnitTests
实现选定的 TC (RED→GREEN)
然后执行 SPEC_reviewImplUnitTests
 19. SPEC_implProductCodes
实现产品代码以使测试通过
 20. SPEC_reviewProductCodes
审查实现质量；提交前重新运行
SPEC_reviewImplUnitTests
SPEC_refactUnitTests (可选) 清理单个 GREEN 测试，
然后再次审查
- 若审查 FAIL：
21. SPEC_abortUserStory (当继续推进不安全时)
将活跃故事移至 abortUS 以供后续分析

暂停/恢复是任何已开启且未关闭步骤的全局中断：

SPEC_suspendUserStory → suspendUS/，附持久恢复引用

SPEC_resumeUserStory → 回到 doingUS/ 并继续

22. SPEC_commitWorks
准备并提交已完成的工作
23. SPEC_closeUserStory
将已审查、已提交的故事从 doingUS/ 移至 doneUS/
将配对的任务工作制品从 doingUS/ 移至 doneUS/
24. SPEC_mergeWorks (当使用了专属故事分支时)
将已关闭的分支合并到集成分支
未创建故事分支时自动跳过

SPEC_partialCloseUserStory 拆分活跃故事：已接受部分带可追溯性
关闭，被拒绝/延后的部分连同理由移入 abortUS/。

SPEC_patchOriginalCaTDD (非默认分支) 将有效的已安装项目
CaTDD 元文件改进回写上游。

流程图：完整生命周期



关键 SpecFlow 命令详解

SPEC_initProjectContext

目的：创建项目的共享宪章——每个后续 SPEC 命令都会引用的事实、约束、约定和决策。

记录内容：

- 项目事实：仓库、所有者、主要目的
- 层次模型：methodPrompts → slashCommands → codeAgents → agentSkills
- 已安装的项目表面：`.catdd/` 目录结构
- 稳定约定：文档拆分规则、中英文镜像规则、方法语义边界

- 当前设计决策：P0/P1/P2 类别列表、设计骨架命名规则
- 验证命令：用于验证文档和打包完整性的脚本

输出：`.catdd/spec/projectContext.md`

为什么重要：没有项目上下文，每次 CodeAgent 会话都从零开始。有了项目上下文，agent 只需读取一个文件就能了解项目的规则、约定和当前状态。

SPEC_analyzeIssue

目的：将原始 Issue 或功能请求转化为可追溯的用户故事。

做什么：

1. 从 `pendingNews/` 读取待处理的 Issue/Feature 文件
2. 结合项目上下文和已有故事对输入进行分析
3. 在 `todoUS/` 中生成结构化用户故事，包含：
 - 角色、能力、价值（US 格式）
 - 候选验收标准
 - 风险评估
 - 给开发者的待定问题
4. 将原始输入移至 `analyzedNews/` 以便追溯
5. 不凭空捏造产品意图——将不明确的方面作为待定问题保留

输出：`todoUS/` 中的用户故事文件和 `analyzedNews/` 中的归档原始输入。

CoT 模式：ReACT（推理 + 行动）——命令必须检查当前状态、决定提取什么、通过编写故事采取行动，并迭代地观察结果。

SPEC_makePlan

目的：为已开启的用户故事创建执行计划。

做什么：

1. 读取 `doingUS/` 中的活跃故事
2. 将工作分类为以下之一：
 - 意图澄清类：开发者与 CodeAgent 的意图尚未对齐
 - 需求导向类：需要更新需求文档
 - 设计导向类：需要架构或详细设计（初始或跟进）
 - 实现导向类：可进入测试设计和代码实现
3. 创建包含 Markdown 复选框任务的 `*-TASKs.md` 文件
4. 根据分类决定下一个 SPEC 命令

输出：`doingUS/YYYYMMDD-TASKs.md` 中的任务工作制品。

为什么重要：这是防止过早设计和实现的规划步骤。没有它，团队往往直接跳到编码，而他们本应对齐意图或先设计架构。

SPEC_clearStoryIntent

目的：在开始设计之前，对齐开发者对故事的理解与 CodeAgent 的理解。

记录内容（"双向意图契约"）：

- 开发者意图：开发者认为此故事是关于什么的
- CodeAgent 意图：agent 从阅读故事中推断出的内容
- 范围内工作：包含什么
- 范围外工作：明确排除什么
- 成功信号：我们如何知道故事已完成
- 假设条件：我们当前假设但尚未确认的内容
- 待定问题：需要开发者回答的问题

为什么重要：如果意图不清，agent 可能会针对错误的范围或成功信号进行优化。一个解决了错误问题的昂贵架构设计，比完全没有设计更糟糕。

SPEC_takeArchDesign 和 SPEC_takeDetailDesign

目的：在两个抽象层次上产出设计工作制品。

架构设计 (SPEC_takeArchDesign)：

- 高层模块分解
- 依赖方向和数据流
- 运行时部署与关键权衡
- 创建 README_ArchDesign.md 和其他架构导向的 SPEC 文档

详细设计 (SPEC_takeDetailDesign)：

- 类级 API 签名和数据结构
- 状态机和生命周期转换
- 活跃故事的验收标准
- 创建 README_DetailDesign.md 和其他详细设计导向的 SPEC 文档

模型层级：这两类命令的初始设计需要 SOTA 推理能力（决定边界、权衡、约束）。跟进修订（SPEC_updateArchDesign、SPEC_updateDetailDesign）可使用 High Performance 模型。

重要：SPEC_take*Design 用于初始设计工作。SPEC_update*Design 针对现有设计证据、审查反馈或故事级设计差距的跟进修订。不要混用。

SPEC_reviewArchDesign 和 SPEC_reviewDetailDesign

目的：在下游生命周期步骤之前把关设计质量。

检查内容：

- 完备性：所有必要的设计方面是否都已覆盖？
- 可追溯性：设计是否与需求关联？
- 一致性：是否存在内部矛盾？
- 可行性：在给定约束下设计是否可实现？

- 清晰性：开发者阅读后能否正确实现？

关键规则：每个产生设计的步骤（SPEC_take*Design、SPEC_update*Design）必须在下游生命周期步骤之前经过其审查关卡的把关。永远不要跳过审查。

SPEC_designUnitTests

目的：进入 CaTDD 测试设计模式，通常经 P0/P1/P2 流程进行。

做什么：

1. 确定测试方法（默认：CaTDD）
2. 根据故事和设计确定需要哪些测试类别
3. 路由到对应的 UT 流程命令
4. 可能会调用 UT_designTypicalSkeleton、UT_designEdgeSkeleton 等

测试方法默认值：CaTDD 是 SpecFlow 中默认的单元测试方法。当项目明确要求时，也可以使用典型 TDD 或其他项目特定的方法。

SPEC_commitWorks

目的：准备并提交已完成的工作，附带可追溯的提交信息。

做什么：

1. 审查变更内容：测试文件、产品代码、设计文档、Spec 工作制品
2. 按故事/关注点对变更进行分组
3. 起草引用 US ID 的提交信息
4. 以适当范围提交
5. 不执行 push——推送是开发者独立的操作

SPEC_closeUserStory

目的：将已完成的工作移至 done 归档。

做什么：

1. 验证故事确实已完成（审查、测试、提交均已完成）
2. 将故事从 doingUS/ 移至 doneUS/
3. 将配对的任务工作制品一并移入
4. 记录关闭日期和所有经验教训
5. 不关闭未经审查和提交的故事

SPEC_updateProjectContext、SPEC_suspendUserStory、SPEC_resumeUserStory、SPEC_mergeWorks 与 SPEC_partialCloseUserStory

- SPEC_updateProjectContext —— 项目事实、约束或约定变化时刷新共享项目宪章；合并改变了生命周期或项目上下文事实后也应运行。
- SPEC_suspendUserStory —— 外部依赖或环境阻塞时，在任何已开启且未关闭的步骤暂停活跃故事。故事与任务制品移至 suspendUS/，附持久恢复引用（分支或 worktree）。

- `SPEC_resumeUserStory` —— 验证恢复引用仍然存在后，将暂停的故事及其任务制品移回 `doingUS/`，然后通过 `SPEC_whatsNextTask` 继续。
- `SPEC_mergeWorks` —— 在 `SPEC_closeUserStory` 之后、仍需分支集成时，将已关闭的故事分支合并到目标集成分支；未使用专属故事分支时自动跳过。
- `SPEC_partialCloseUserStory` —— 当活跃故事只有部分范围被接受时拆分：已接受部分带可追溯性关闭，被拒绝或延后部分连同显式理由与证据移入 `abortUS/`。

`SPEC_whatsNextTask` 可随时调用，读取当前 `.catdd/spec/` 状态并推荐唯一下一步命令。

命令的模型层级指引

Px-SpecFlow 提供了明确的模型层级指引——使用能够保持决策质量的最小模型：

层级	用途	SPEC 命令
SOTA reasoning	架构决策、系统边界、质量权衡、不可逆选择	<code>SPEC_takeArchDesign</code> 、 <code>SPEC_reviewArchDesign</code>
High Performance	需求分析、意图对齐、规划、需求更新、局部设计、审查关卡、测试设计、代码审查、纠正路由、受控上游补丁回写	<code>SPEC_initProjectContext</code> 、 <code>SPEC_updateProjectContext</code> 、 <code>SPEC_analyzeIssue</code> 、 <code>SPEC_analyzeFeature</code> 、 <code>SPEC_analyzeAbortedUserStory</code> 、 <code>SPEC_clearStoryIntent</code> 、 <code>SPEC_makePlan</code> 、 <code>SPEC_updateUserStory</code> 、 <code>SPEC_whatsNextTask</code> 、 <code>SPEC_takeArchDesign</code> 、 <code>SPEC_reviewArchDesign</code> 、 <code>SPEC_updateArchDesign</code> 、 <code>SPEC_takeDetailDesign</code> 、 <code>SPEC_reviewDetailDesign</code> 、 <code>SPEC_updateDetailDesign</code> 、 <code>SPEC_reviewUserStory</code> 、 <code>SPEC_designUnitTests</code> 、 <code>SPEC_reviewImplUnitTests</code> 、 <code>SPEC_reviewProductCodes</code> 、 <code>SPEC_patchOriginalCaTDD</code>
Flash Speed	确定性的导入、移动、暂停、恢复、部分关闭、中止、提交、关闭，或小型测试驱动实现/重构步骤	<code>SPEC_importIssue</code> 、 <code>SPEC_importFeature</code> 、 <code>SPEC_importUserStory</code> 、 <code>SPEC_openUserStory</code> 、 <code>SPEC_suspendUserStory</code> 、 <code>SPEC_resumeUserStory</code> 、 <code>SPEC_partialCloseUserStory</code> 、 <code>SPEC_abortUserStory</code> 、 <code>SPEC_implUnitTests</code> 、 <code>SPEC_implProductCodes</code> 、 <code>SPEC_refactUnitTests</code> 、 <code>SPEC_commitWorks</code> 、 <code>SPEC_closeUserStory</code>

升级规则：当命令暴露出架构层面显著的不确定性时，从较低层级升级到较高层级：竞争性非功能需求、安全/保密风险、实时或嵌入式约束、并发边界、数据迁移、兼容性矩阵，或不可逆的模块/API 所有权决策。

命令中的思维链模式

SPEC 命令使用三种 CoT 推理模式，根据命令的决策复杂度进行选择：

ReACT（推理 + 行动）

用于必须检查当前生命周期状态、决定做什么、采取行动并迭代地观察结果的命令。适用于分析、设计和审查命令。

执行循环：

1. 思考 (Thought)：检查输入工作制品。识别当前状态、缺口和冲突。
2. 行动 (Action)：创建、更新、审查或移动命名的工作制品。
3. 观察 (Observation)：验证输出是否满足契约。检查遗漏的可追溯性、未解决的问题、质量失败。
4. 如果观察发现质量问题，将其作为问题或假设提出并停止。

适用命令：SPEC_analyzeIssue、SPEC_makePlan、SPEC_reviewArchDesign、SPEC_reviewDetailDesign、SPEC_reviewProductCodes

ToT（思维树）

用于命令必须生成多个候选方案、根据质量标准评估每个方案并选择最佳方案的场景。

执行循环：

1. 生成 (Generate)：生成 2 个以上候选方案或工作制品大纲。
2. 评估 (Evaluate)：根据质量标准评估每个方案。
3. 选择 (Select)：选择最佳方案。如果没有明显最优的方案，向开发者呈现选项。
4. 执行 (Execute)：应用选定的方案产出工作制品。
5. 验证 (Verify)：确认输出满足契约。

适用命令：SPEC_takeArchDesign（比较架构替代方案）、SPEC_clearStoryIntent（评估不同范围）

Linear（直接执行）

用于当给定完整且有效的输入时，命令的动作是确定性的。

执行循环：

1. 读取输入和方法引用。
2. 保留现有的可追溯性、状态标记和约定。
3. 仅执行此命令所请求的步骤。
4. 报告产出的工作制品和下一个推荐命令。

适用命令：SPEC_importIssue、SPEC_openUserStory、SPEC_commitWorks、SPEC_closeUserStory

SpecFlow 工作制品

SpecFlow 创建并维护一组可追溯的工作制品：

生命周期工作制品（位于 `.catdd/spec/` 下）

工作制品	用途	Git 策略
<code>projectContext.md</code>	共享项目宪章	提交——团队成员和 agent 需要相同事实
<code>pendingNews/*.md</code>	已导入、等待分析的工作	提交——团队可见性
<code>analyzedNews/*.md</code>	分析后归档的原始输入	提交——保留可追溯性
<code>todoUS/*-UserStory.md</code>	已分析、可领取的故事	提交——团队待办列表
<code>doingUS/*-UserStory.md</code>	正在进行中的活跃故事	提交——跨机器可见性
<code>doingUS/*-UserStory-Tasks.md</code>	活跃任务计划，含复选框任务	提交——显式、可检查的步骤
<code>suspendUS/*-UserStory.md</code>	暂停的故事，附持久恢复引用	提交——保留可追溯性
<code>suspendUS/*-UserStory-Tasks.md</code>	与故事并存的暂停任务工作制品	提交——下一步恢复保持显式
<code>abortUS/*-UserStory.md</code>	为重新分析保留的中止故事	提交——保留可追溯性
<code>abortUS/*-UserStory-Tasks.md</code>	与故事并存的中止任务工作制品	提交——供后续分析或下一轮使用
<code>doneUS/*-UserStory.md</code>	已完成、已审查、已提交的故事	提交——项目历史
<code>doneUS/*-UserStory-Tasks.md</code>	已完成的任务工作制品	提交——后续诊断
<code>WorkingProcessLog.md</code>	本地工作状态跟踪	Gitignore——个人的，非团队共享

项目根 SPEC 文档

文件	用途	管理者
README.md	项目概览、所有权、主目录	其他 SPEC 步骤
README_ArchDesign.md	高层架构、模块、依赖关系	SPEC_takeArchDesign
README_DetailDesign.md	类设计、API 签名、数据结构	SPEC_takeDetailDesign
README_UserStories.md	项目级 TODO/DONE 故事账本，含 AC 追溯/状态	SPEC_updateUserStory、SPEC_reviewUserStory
README_UserGuide.md	面向用户的运行时使用指南	SPEC_updateUserStory、SPEC_reviewUserStory
README_VerifyDesign.md	验证拓扑、测试策略、US/AC/TC 可追溯性	SpecFlow + UT 流程
README_ErrorDesign.md	容错架构、故障安全状态	SPEC_takeArchDesign
README_ResourceDesign.md	资源分配、内存/CPU 预算	SPEC_takeArchDesign
README_StateDesign.md	状态机、生命周期转换	SPEC_takeDetailDesign
README_PerfDesign.md	性能预算、延迟限制	SPEC_takeArchDesign
README_CompatDesign.md	兼容性矩阵、平台/工具链版本	SPEC_takeArchDesign
README_DiagnosisDesign.md	可观测性、日志、遥测、诊断	SPEC_takeArchDesign
README_UsageDesign.md	公共边界、CLI/API 契约	SPEC_takeArchDesign

仅当项目需要该表面时才创建项目根 SPEC 文档。首次创建时，使用 `slashCommands/templates/` 中的模板。

命令模板即契约

每个 slash 命令都是开发者与 CodeAgent 之间的契约。开发者调用命令（表明意图）。CodeAgent 执行契约（忠实地应用方法规则）。该契约包含：

1. 前置条件：调用前必须存在的输入（例如 API 头文件、已有骨架、项目上下文）
2. 动作：命令执行的单一步骤（不是“设计、实现和审查”——只选其一）
3. 后置条件：执行后必须存在的输出（例如设计好的骨架、已实现的测试、审查报告）
4. 不变量：必须保留的内容（CaTDD 评论骨架、US/AC/TC 可追溯性、类别标签、状态标记）
5. 下一步：接下来应该执行什么命令（永远不要让 CodeAgent 猜测）

这种契约结构正是 slash 命令可重复的原因。每次调用 `UT_designTypicalSkeleton` 都会产生一个 CaTDD Typical 骨架，无论由哪个 CodeAgent 执行，因为契约精确定义了输出。

冲突保护规则

Px-SpecFlow 定义了严格的冲突保护规则，防止各层违反彼此的边界：

1. Px-SpecFlow 仅定义生命周期编排；CaTDD 方法语义保留在 methodPrompts 中。
2. SPEC 命令可以调用 UT 命令，但不得替换 P0/P1/P2 类别规则。
3. 在需求导向工作中，SPEC_updateUserStory 之后不得跳过 SPEC_reviewUserStory。
4. 当活跃故事的开发者意图和 CodeAgent 意图尚未对齐时，不得开始设计。
5. 在 SPEC_makePlan 之后，SPEC_take*Design 仅用于初始设计工作，SPEC_update*Design 仅用于跟进设计修订。
6. 每个产生设计的步骤必须在下游生命周期步骤之前经过其审查关卡的把关。
7. 如果产品意图不明确，保持用户故事开放并询问开发者，而不是凭空捏造需求。
8. HARNESS_* 命令是运维工具点：它们不得移动 SpecFlow 生命周期状态，也不得重新定义 CaTDD 类别含义。
9. 在专属故事分支上执行 SPEC_closeUserStory 之后，若仍需分支集成，请运行仓库合并/集成步骤（例如 SPEC_mergeWorks）；未创建分支则跳过合并。

执行模式与 ONE-MORE-THING 停止规则

Px-SpecFlow 无论以交互式还是无人值守方式运行，都执行完全相同的流程：

- manualMode（默认）：交互式聊天。一次执行一个 slash 命令；当意图、标准或安全性不清晰时，助手提出聚焦问题。
- autonomousMode（可选加入）：通过 specCodeAgentCLI 或带 execution_mode: autonomousMode 的入口命令进行连续的无头/CLI 执行。agent 借助显式文件工作制品自动执行并推进安全的下一步，记录假设而不在每个回合停下。

安全边界：只有实现导向的故事支持 autonomousMode。若在意图澄清、需求导向或设计导向的故事上触发自主模式，流程必须停止、强制 manualMode 并要求开发者确认。需求分析与系统架构需要人的意图与权衡。

命令级的 analysis_mode（默认 BRAINSTORM，或 AUTONOMOUS）是 SPEC_analyzeIssue / SPEC_analyzeFeature 内部的局部设置；它不会把流程切换到 autonomousMode。

每个 CaTDD 命令都执行 ONE-MORE-THING 通用停止规则：不确定时询问开发者。在 manualMode 下，助手立即停止并提问。在 autonomousMode 下，自主绝不是猜测的许可——运行必须停止，输出 status: manual_required: ONE-MORE-THING: <question> 并等待。

自主运行在完成（SPEC_closeUserStory → 退出码 0）、中止（SPEC_abortUserStory → 非零退出码）或暂停（SPEC_suspendUserStory → 干净退出）时终止。

实践中的命令调用

Slash 命令根据不同的 CodeAgent 以不同方式调用：

在 Copilot Chat 中

```
/UT_designTypicalSkeleton
```

Copilot 提示词包装器（由 `scripts/makeSlashCmd4Copilot.sh` 生成）将可移植的 Markdown 命令适配为 Copilot 的提示词格式。开发者输入 slash 命令，Copilot 读取适配后的提示词，并执行契约。

在 Cline/Continue 中

直接读取命令提示词文件。开发者输入命令名称或从助手的命令面板中选择。

在 utCodeAgentCLI 中

```
utCodeAgentCLI --target myTestFile.ts --input spec/I0C.h --behave designTypicalSkeleton
```

CLI 将 `--behave` 的值映射到 slash 命令行为。别名如 `reviewFuncTestsSkeleton` 和 `tellMeNextImplTest` 会解析为其完整的 UT 命令名称。

可移植占位符

命令文件使用适配器填充的可移植占位符：

- `{{feature_name}}` —— 被测试功能/模块的名称
- `{{category}}` —— CaTDD 类别（Typical、Edge 等）
- `{{source_files}}` —— API 头文件或规格文档
- `{{test_files}}` —— 要读取和扩展的已有测试文件
- `{{language}}` —— 目标编程语言
- `{{test_framework}}` —— 使用的测试框架
- `{{developer_goal}}` —— 未在工作制品中体现的开发者意图

何时使用哪个流程

起点	使用流程
新组件，无现有测试	P0-FuncTestsFlow → <code>UT_designTypicalSkeleton</code>
现成的 Demo 代码	P0-FuncTestsFlow → <code>UT_convertDemoToTypical</code>
完整功能测试设计	P0-FuncTestsFlow → <code>UT_designFuncTestsSkeleton</code>
需要架构验证	P1-DesignTestsFlow（在 P0 完成后）
需要性能/容量数据	P2-QualityTestsFlow（在 P0/P1 完成后）
来自产品团队的新功能请求	Px-SpecFlow → <code>SPEC_importFeature</code> → <code>SPEC_analyzeFeature</code>
来自 QA 的 Bug 报告	Px-SpecFlow → <code>SPEC_importIssue</code> → <code>SPEC_analyzeIssue</code>
已有用户故事	Px-SpecFlow → <code>SPEC_importUserStory</code> → <code>SPEC_openUserStory</code>
不知道下一步做什么	Px-SpecFlow → <code>SPEC_whatsNextTask</code> （读取当前状态并推荐唯一下一步）
恢复已暂停的故事	Px-SpecFlow → <code>SPEC_resumeUserStory</code> （验证持久恢复引用）

起点	使用流程
需要架构决策	Px-SpecFlow → SPEC_takeArchDesign
设计完成后准备编码	Px-SpecFlow → SPEC_designUnitTests → P0/P1/P2 流程
已安装的 CaTDD harness 疑似损坏	Px-HarnessKits → HARNESS_verifyInstallation → HARNESS_diagnoseInstallation
已验证运行中的可复用经验	Px-HarnessKits → HARNESS_evolveHarness

子 Agent 模式

某些 SPEC 命令支持通过子 agent 进行后台委托。当某个命令适合在对话过程中委托执行时：

- 何时委托：当其他工作正在进行时出现新 Issue
- 捕获内容：源文本或 URL 及相关的项目上下文
- 期望返回：输出文件名（例如 todoUS/ 中的新故事）
- 调用方行为：主对话继续，无需等待

适合子 agent 委托的命令：SPEC_importIssue、SPEC_importFeature、SPEC_analyzeIssue、SPEC_analyzeFeature。

必须同步执行的命令：SPEC_takeArchDesign（输出需要立即用于审查）、SPEC_makePlan（决定下一步）、SPEC_commitWorks（必须与开发者的工作空间同步）。

从命令到自动化

Slash 命令是 VibeCoding 对话式灵活性与 CodeAgent 自动化之间的桥梁。当开发者调用 /UT_implTestCase 时，CodeAgent 读取命令契约，应用 CaTDD 方法，并产生结构化输出。当开发者调用 /SPEC_analyzeIssue 时，CodeAgent 读取原始 Issue，将其转化为用户故事，并使其在生命周期中流转。

下一章 asyncCodeAgent 将更进一步：CaTDD 原生代码 agent 不再等待开发者逐个调用命令，而是自主规划、执行、收集追踪并反思——将整个循环从目标到经过验证的实现完全自动化。

asyncCodeAgent

devBooks/ActionZH/04_asyncCodeAgent.md

什么是 Code Agent ?

代码 agent (Code Agent) 是一种自主执行开发任务的软件工具。它读取方法规格、规划工作、调用命令、生成代码、运行测试、收集结果，并将经验教训反馈回方法论体系中。它不是一个聊天机器人——它是一个无需持续人工提示即可运行的执行引擎。

在 CaTDD 中，代码 agent 是四层架构中的第三层。它位于 methodPrompts (定义做什么) 和 slashCommands (定义如何调用步骤) 之下，以及 agentSkills (为其他 agent 封装能力) 之上。代码 agent 是智能执行层——它闭合了从可复用的 CaTDD 知识到规范化的 CaTDD 原生执行之间的循环。

两种代码 Agent

MyCaTDD 定义了两类职责各异的代码 agent :

utCodeAgentCLI —— 单元测试 Agent

utCodeAgentCLI 是 CaTDD 原生的单元测试 agent。它专注于一个关注点：将开发者目标转化为经过验证的单元测试。

做什么：

- 接收开发者目标 (例如"为 EventQueue API 设计和实现 P0 功能测试")
- 依据 CaTDD 方法约束 (优先级顺序、类别规则、质量关卡) 规划工作
- 调用标准化的 slash 命令步骤 (UT_designTypicalSkeleton、UT_implTestCase 等)
- 收集执行追踪记录 (运行了什么、通过了什么、失败了什么、被阻塞了什么)
- 反思结果 (结果是否正确? 是否发生了意外情况?)
- 将可复用模式反馈给 methodPrompts 和 slashCommands

不做什么：

- 不决定产品意图——由开发者掌控
- 不重新定义 CaTDD 方法语义——由 methodPrompts 掌控
- 不管理 SpecFlow 生命周期——由 specCodeAgentCLI 掌控 (或由开发者直接驱动)

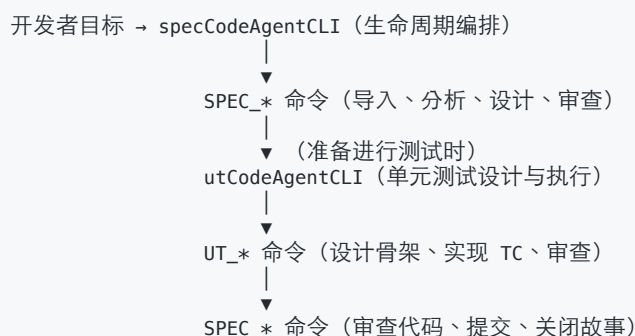
specCodeAgentCLI —— 规格 Agent

specCodeAgentCLI 是面向 SpecCoding 的 agent。它专注于模块级从输入到输出的流转——从 Issue 导入到故事关闭的完整生命周期。

做什么：

- 基于 Px-SpecFlow 生命周期规则运作
- 在测试设计和实现阶段复用 utCodeAgentCLI 的单元测试优势
- 组织从规格意图到可执行检查的场景级验证
- 保持 Spec 流程、验证检查点和实现结果之间的可追溯性

工作管线：



两个 agent 共同自动化了从"我们有一个功能请求"到"测试通过、代码已提交、故事已关闭"的完整路径。

CaTDD 原生契约

CaTDD 原生的代码 agent 不是通用的 LLM 封装器。它与 CaTDD 方法论之间有一个明确的契约：

1. 保留评论骨架契约 —— agent 必须读取、保留和更新 `@[US]`、`@[AC]`、`@[TC]`、`@[Category]` 和状态标记。这些是结构性的，而非装饰性的。
2. 遵循优先级纪律 —— agent 必须先在 P0 完成之后再行 P1，先在 Typical 完成之后再行 Edge，并尊重上下文相关的优先级调整和风险评分。
3. 维护 US/AC/TC 可追溯性 —— 每个测试用例必须追溯到一条验收标准，每条验收标准必须追溯到一个用户故事。agent 必须在跨文件场景下保留这些链接。
4. 尊重质量关卡 —— agent 必须在每个关卡处停止，并在推进到下一优先级级别之前验证标准。不允许跳过关卡。
5. 遵守状态标记 —— agent 必须保持 `○→●→●` 标记准确无误，并在测试文件和 TODO 跟踪区间保持同步。
6. 向上游层次反馈 —— 当 agent 发现一个应该可复用的模式时，将其反馈给 slashCommands（作为新的命令流程）和 methodPrompts（作为方法论的改进）。

Agent 工作流程检查清单

CaTDD 方法提示词定义了一个包含四个阶段的完整 Agent 工作流程检查清单。这是每个 CodeAgent 的执行脚本：

阶段 1：理解（只读分析）

在任何设计或实现之前，agent 必须：

1. 读取组件接口文件 —— 定位并读取 API 头文件，识别公共函数、数据结构、常量、签名和返回类型。

2. 研究现有的相关测试 —— 搜索已有的测试文件，审查相似的测试模式，识别可复用的 fixture 和辅助函数。
3. 识别依赖关系和约束 —— 检查构建文件中的依赖关系，审查设计文档，必要时阅读源码实现，注意特殊的构建要求。
4. 与开发者澄清模糊之处 —— 如果 API 行为不明确，提出具体问题。如果需求模棱两可，提出替代方案。如果上下文不充分，请求具体的文件。

检查点 1 输出：

"我已分析了 [组件]。它提供 [关键能力]。

审查的文件：

- 接口：Include/[HeaderFile.h]
- 实现：Source/[SourceFile.c]
- 已有测试：Test/UT_[Related].cxx
- 文档：README_[Topic].md

关键 API：[列出 3-5 个主要函数]

依赖关系：[列出主要依赖项]

不明确的方面：[如有，列出问题]

是否继续进入测试设计？"

Agent 在此处停止。在开发者确认理解正确之前，不推进到设计阶段。

阶段 2：设计（编写注释——尚无代码）

Agent 在结构化注释中创建全面的测试设计：

1. 填充 OVERVIEW 区域 —— WHAT、WHERE、WHY、SCOPE（范围内 vs 范围外）、KEY CONCEPTS。
2. 定义覆盖矩阵维度 —— 识别 2-3 个关键维度以实现系统化覆盖，创建展示维度组合的表格，将组合映射到潜在的用户故事。
3. 编写用户故事（2-5 个） —— 作为 [角色]，我想要 [能力]，以便 [价值]。每个故事应独立有价值。覆盖成功和错误两种场景。
4. 编写验收标准（每个 US 2-4 条） —— GIVEN/WHEN/THEN 格式，可独立测试，对预期行为和错误码描述具体。
5. 细化测试用例（每条 AC 至少 1 个） —— 名称、目的、简述、步骤、期望、备注。使用优先级框架进行分类。全部标记为 ☐ TODO。
6. 填充 TODO 跟踪区 —— 列出所有计划中的测试用例，包含状态、优先级指示器和所有依赖/阻塞说明。

检查点 2 输出：

"[组件] 的测试设计已完成：

- 覆盖范围：[X] 个用户故事，[Y] 条验收标准，[Z] 个测试用例
- 优先级分布：P0=[数量]，P1=[数量]，P2=[数量]
- 已覆盖的关键场景：[列出 3-5 个主要场景]
- 预估实现工作量：[粗略估算]

是否继续进入实现阶段？"

Agent 在此处也停止。设计审批必须先于实现。

阶段 3：实现（TDD Red→Green 循环）

这是最详细的阶段，因为它覆盖了按优先级级别执行的完整 Red→Green 循环。

3A : Fast-Fail Six (快速验证)

Agent 优先实现以下六个测试，因为它们能尽早捕获 API 契约违规：

1. Null/Empty 输入处理
2. Zero/Negative 超时
3. Duplicate 注册/订阅
4. 非法调用顺序 (init 前、cleanup 后)
5. Buffer full/empty 边界
6. Double-close/re-init 幂等性

全部六个标记为 ● RED，运行，确认全部失败，报告。

3B : P0 功能 —— ValidFunc (Typical + Edge)

Agent 依次完成 Typical 然后 Edge 测试：

- 实现测试 (四阶段：SETUP/BEHAVIOR/VERIFY/CLEANUP)
- 标记为 ● RED
- 运行，确认 RED (失败)
- 实现最小量产品代码
- 运行，确认 GREEN (通过)
- 标记为 ● GREEN
- 提交
- 移至下一个测试

3C : P0 功能 —— InvalidFunc (Misuse + Fault)

同样的循环，先 Misuse 后 Fault 测试。

关卡 P0 检查点：在推进到 P1 之前，agent 必须验证：

- 所有 P0 ValidFunc 测试为 GREEN (Typical + Edge)
- 所有 P0 InvalidFunc 测试为 GREEN (Misuse + Fault)
- Fast-Fail Six 全部通过
- 被测试模块的代码覆盖率 $\geq 80\%$
- 无内存泄漏 (使用 sanitizer 运行)
- 无关键功能 Bug

Agent 报告关卡状态并询问："P0 完成。是否继续 P1？"

3D : P1 设计导向测试 (如适用)

State → Capability → Interaction → Concurrency，遵循相同的 RED→GREEN 循环。

关卡 P1 要求：ThreadSanitizer/AddressSanitizer 干净，无死锁，无竞态条件，交互/协作契约已对照其设计源验证，架构按设计要求验证通过。

3E：P2 质量导向测试（如需要）

Performance → Robust → Compatibility → Configuration → Diagnosis → Security。

关卡 P2 要求：性能 SLO 已满足，压力/浸泡测试通过，需要时诊断/可观测性证据已验证，存在威胁模型或策略时安全保护测试 GREEN，生产就绪标准达标。

阶段 4：收尾与文档

Agent：

1. 重构测试以提升清晰度（提取公共 fixture、移除重复、简化）
2. 更新文档（确保注释反映实际实现，清除过时的 TODO 条目）
3. 记录已知限制或问题
4. 将所有已完成的测试标记为 ● GREEN
5. 提供完成报告

最终检查点：

```
"[组件] 的测试已完成：
✓ 已实现测试：[数量] (P0=[n], P1=[n], P2=[n])
✓ 测试覆盖率：[百分比]%
✓ 全部测试通过：[是/否]
▲ 已知问题：[如有，列出]
○ 阻塞项：[如有，列出]

下一步：[建议]"
```

Agent 故障排除：六大常见问题

CaTDD 方法提示词为 agent 定义了故障排除指南。六大问题覆盖了最常见的失败模式：

问题 1：测试编译失败

Agent 解决方式：

1. 根据项目的 include 模式检查 #include 语句
2. 根据头文件验证函数签名（而非假设）
3. 检查缺失的测试工具函数
4. 向开发者反馈具体错误详情以及已检查过的内容

问题 2：测试设计似乎不完整或有误

Agent 解决方式：

1. 验证对齐性：TC → AC → US 可追溯链
2. 检查覆盖矩阵的完备性（预期 vs 实际场景）
3. 根据已知行为验证测试期望
4. 审查 Fast-Fail Six 清单中遗漏的覆盖项

问题 3：产品代码行为不明确

Agent 解决方式：

1. 在代码库中搜索相似模式（错误码、AC 示例、命名模式）
2. 按顺序阅读组件文档：spec → 架构 → 术语表 → 设计文档 → 源码注释
3. 检查已有测试中的行为规格
4. 向开发者提供 2-3 个具体备选方案（而非开放式地提问“这应该怎么做？”）

问题 4：测试意外失败

Agent 解决方式：

1. 验证测试设置是否正确（初始化、前置条件、资源创建）
2. 添加诊断输出（打印实际值 vs 预期值、中间状态）
3. 检查测试隔离性（先前测试的清理情况、全局状态污染）
4. 向开发者报告发现结果，附带具体的预期值/实际值和诊断输出

问题 5：无法继续 / 被阻塞

Agent 解决方式：

1. 明确说明阻塞因素：缺少什么、影响、已考虑的权宜方案
2. 在 TODO 区域使用  BLOCKED 标记记录，注明依赖关系和预估工作量
3. 提出具体的后续步骤，包含选项（例如：先实现缺失的 API、推迟被阻塞的测试、使用硬编码常量并标记 TODO）
4. 继续未阻塞的工作——绝不空闲等待

问题 6：测试在应该失败时却通过了（RED 阶段）

Agent 解决方式：

1. 验证测试确实被执行（添加 printf、添加临时失败断言）
2. 检查该功能是否已存在（产品代码是否在预期位置？）
3. 验证测试断言是否有实际意义（不仅仅是“发生了某事”）
4. 必要时更新测试设计——如果功能已完成则标记 GREEN，如果测试太弱则增强

Agent 的“该做与不该做”契约

方法提示词为 agent 定义了明确的行为规则：

该做（DO）：

- 尽早提出澄清问题（阶段 1）——在设计开始之前
- 在检查点处等待人工审批——绝不自行推进
- 每次测试操作后立即更新 TODO 区域——状态必须反映实际
- 严格遵循 RED→GREEN 纪律——绝不跳过 RED 阶段

- 每次达成 GREEN 后提交——小规模、可追溯的提交
- 频繁运行测试，立即报告失败——避免在最后出现意外

不该做 (DON'T) :

- 不经设计直接跳到实现——设计就是评论骨架
- 在编写测试之前实现产品代码——违反 TDD
- 让测试一直保持 RED 而不处理——RED 意味着"需要行动"
- 将多个功能批量放入一个测试——一个 TC，一个行为
- 凭空猜测需求——改为提出询问
- 在完成 P0 之前实现 P1/P2——优先级纪律是不可妥协的

追踪收集与反思

CaTDD 代码 agent 不仅仅是一个执行器——它是一个学习者。在完成一个会话后，它会收集：

执行追踪

- 调用了哪些命令及其顺序
- 读取和写入了哪些文件
- 设计和实现了哪些测试用例
- 哪些通过了 (GREEN)，哪些失败了 (RED)，哪些被阻塞 (🚫)
- 遇到了哪些错误以及如何解决
- 向开发者提出了哪些问题以及收到了哪些答复

反思记录

- 是否存在跨多个测试文件重复出现的模式？
- 是否有对该组件而言不需要的类别？
- 事后回想，是否有本应更早优先处理的类别？
- 开发者的回复是否持续暗示方法提示词存在缺口？
- 某个故障排除方式是否应该成为 agent 的标准行为？

反馈循环

Agent 将反思记录反馈给上游层次：

Agent 发现模式 → 规范化为 slash 命令 → 添加到 slashCommands/
Agent 发现方法缺口 → 改进方法提示词 → 更新 methodPrompts/
Agent 发现可复用技能 → 封装为 agent 技能 → 更新 agentSkills/

这就是双向改进循环的实际运作。每次 agent 执行都会为未来所有执行改进方法论。

utCodeAgentCLI 架构

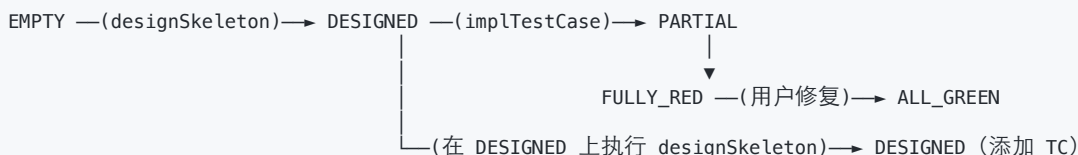
在当前仓库阶段，`utCodeAgentCLI` 是一份已文档化的架构和设计契约——可运行的 CLI 实现正在进行中。该架构将两个系统分开：

1. `AgentSDK`：一个通用的 LLM agent 运行时库。它了解目标、消息、工具、会话、权限、追踪、钩子、适配器和执行控制。它不了解 `CaTDD`。
2. `utCodeAgentCLI`：构建于 `AgentSDK` 之上的 `CaTDD` 应用程序。它解析 CLI 参数，将 `CaTDD` 行为解析为可移植的 slash 命令，注入方法提示词引用，保留 `US/AC/TC` 可追溯性，并记录 `CaTDD` 执行追踪。它是一个编排器，而非方法所有者。

这种分离是架构中最重要的风险缓解措施：一个便捷的 CLI 很容易滑向重复 `CaTDD` 语义、自行定义类别含义或绕过可移植 slash 命令契约。通过保持 `AgentSDK` 通用化、`utCodeAgentCLI` 轻薄化，架构防止了方法漂移。

测试文件状态模型

`utCodeAgentCLI` 通过正式的状态模型跟踪每个测试文件：



文件状态	描述
EMPTY	文件中不存在 <code>CaTDD</code> 骨架 TC
DESIGNED	所有 TC 为 <code>@[Status:PLANNED]</code> — 设计完成，尚未实现
PARTIAL	混合 <code>PLANNED</code> 、 <code>RED</code> 和 <code>GREEN</code> — 部分已实现，部分未实现
FULLY_RED	所有 TC 为 <code>RED</code> 或 <code>GREEN</code> ，无 <code>PLANNED</code> — 全部测试代码已编写
ALL_GREEN	所有 TC 为 <code>GREEN</code> — 测试通过，覆盖达成

CLI 拥有 `PLANNED` → `RED` 的转换（编写测试代码）。`RED` → `GREEN` 的转换是用户拥有的——CLI 读取 `GREEN` 状态但永远不写入它。这保留了 TDD 契约：由人（或人工批准的产品代码）将 `RED` 变为 `GREEN`。

行为状态契约

每个 `--behave` 值都与文件状态有精确的契约：

行为	要求	产出 (TC 状态)	产出 (文件状态)
<code>design*Skeleton</code>	Any	新 TC → <code>PLANNED</code>	<code>DESIGNED</code> 或 <code>PARTIAL</code>
<code>review*Skeleton</code>	<code>DESIGNED</code> , <code>PARTIAL</code> , <code>FULLY_RED</code>	不变	不变
<code>tellMeNextImplTest</code>	至少 1 个 <code>PLANNED</code> TC	不变	不变
<code>implTestCase</code>	目标 TC 为 <code>PLANNED</code>	目标 TC → <code>RED</code>	<code>PARTIAL</code> 或 <code>FULLY_RED</code>
<code>implTestFile</code>	至少 1 个 <code>PLANNED</code> TC	所有 <code>PLANNED</code> → <code>RED</code>	<code>FULLY_RED</code>

行为	要求	产出 (TC 状态)	产出 (文件状态)
designAndImplTest	Any	所有 TC → RED	FULLY_RED

状态保留保证：

1. 绝不降级状态 (RED → PLANNED 永不发生)
2. 绝不在无明确意图的情况下覆盖已实现的 TC
3. 状态不匹配的行为以清晰错误退出
4. 每次状态转换都记录在执行追踪中

核心组件

组件	职责
Parser	解析开发者命令： <code>--goal</code> 、 <code>--target</code> 、 <code>--input</code> 、 <code>--behave</code> 、 <code>--model-tier</code>
Planner	将目标分解为有序的 slash 命令调用序列
Executor	针对 CodeAgent 运行时（模型提供方 + 工具表面）调用 slash 命令
Adapter	将可移植的命令契约适配到特定的模型运行时（Copilot API、Cline 协议、直接 LLM）
Trace	收集结构化执行日志：命令、TC、状态转换、时间戳、错误
Diagnostics	验证执行质量：RED→GREEN 完整性、关卡合规、可追溯性保留
State	管理 agent 会话状态：活跃目标、当前阶段、待处理检查点
Error	处理 agent 错误：模型失败、畸形输出、缺失工作制品、前置条件不满足

CLI 接口设计

```
utCodeAgentCLI \
  --goal "Design and implement P0 functional tests for EventQueue" \
  --target codeAgents/utCodeAgentCLI/SysTests/UT_EventQueue.ts \
  --input spec/EventQueue.h \
  --behave designFuncTestsSkeleton \
  --model-tier high-performance
```

`--target` 参数定义测试空间范围：一个 TestFile 中的一个 TestCase，一个 TestFile，或若干 TestFiles。

`--input` 参数携带源/上下文，如接口、协议、模式、草稿或产品源代码。`--behave` 参数指定一个兼容的 UT slash 命令行为或一个稳定的 CLI 别名。

运行时决策

utCodeAgentCLI 架构设计包含以下决策：

- V1 (PoC)：TypeScript/Node.js —— 快速原型开发，便于为 Copilot 和类似 agent 集成 Adapter SDK
- V2 (生产)：Go —— 编译后单一二进制文件，零运行时依赖，易于分发
- Python：已评估但未选择 —— 并发 agent 会话的性能问题，协议契约的类型检查较弱

用户故事层次结构

utCodeAgentCLI 的需求按角色而非功能组织：

角色	故事数	关注点
USER	10 个故事	引导式发现（NEW-USER）和精准控制（EXPERIENCED-USER）
INVENTOR	3 个故事	方法委托验证、可追溯性完整性、诊断证明
DEVELOPER	5 个故事	错误消息、日志记录、交互模式、适配器接口、可靠性策略执行

USER 旅程：NEW-USER 遵循引导式发现（验证 → 设计所有骨架 → 审查所有层级 → 选择下一个 → 设计与实现）。EXPERIENCED-USER 使用精准控制（验证 → 单一类别设计 → 层级审查 → 实现一个 TC → 审查实现）。两种路径都生成机器可读的执行追踪。

INVENTOR 需求确保 CLI 是 CaTDD 的忠实代理：它必须引用 `methodPrompts` 获取类别语义（绝不硬编码）、必须经过 `slashCommands` 实现可移植执行（绝不绕过）、必须生成结构化的追踪文件以证明这些约束得到了满足。

DEVELOPER 需求涵盖运行时质量：为每种已知失败模式提供确定性错误消息、可配置级别的结构化日志、审查关卡的交互式确认模式（含 CI 安全的非交互回退方案）、针对不同模型运行时的清晰适配器边界，以及可执行的可靠性/安全契约。

非需求（utCodeAgentCLI 不拥有什么）

与 CLI 做什么同样重要的是它明确不做什么：

关注点	所有者
定义 CaTDD 类别、纪律规则或方法含义	<code>methodPrompts/</code>
定义可移植的 slash 命令执行逻辑	<code>slashCommands/</code>
将 CaTDD 封装为通用 CodeAgent 技能	<code>agentSkills/</code>
编译、运行或验证测试代码	用户的构建系统
生成产品/源代码	CLI 仅生成测试代码
管理 git 分支、提交或版本控制	用户的工作流程
将 TC 从 RED 转为 GREEN	用户的 TDD 工作流程

架构层重要需求（ASR）

utCodeAgentCLI 架构定义了六项在架构边界层面运作的可靠性/安全需求：

ASR	需求	信号
ASR-R 1	重试与修正循环在预算耗尽时应为有界且确定性	明确的重试预算拥有者和确定性的耗尽路径
ASR-R 2	未知或不支持的行为路由应为确定性且可诊断	无静默强制转换；明确的诊断回退
ASR-R 3	故障处理应区分暂时性和永久性类别，并具有明确路由	故障分类法和类别特定的控制流

ASR	需求	信号
ASR-R 4	多步执行应定义快照/回滚或补偿边界	步骤边界一致性模型和失败后的变更控制
ASR-R 5	升级策略应定义阈值和非交互行为	明确的升级触发器和 CI 安全的中止动作
ASR-R 6	Shell 执行应强制执行安全策略和敏感路径保护	白名单执行模型、敏感路径把关、脱敏策略

这些 ASR 可追溯到 DEVELOPER 需求 (US-DEV-05) 中的可执行 US/AC 契约。每个 ASR 都映射到具体的验收标准——它们不是空洞的愿景声明，而是产出可验证验收信号的架构层需求。

架构决策记录 (ADR)

关键架构决策以 ADR 形式被正式记录：

ADR：运行时语言 —— utCodeAgentCLI 采用阶段性运行时决策：

- V1 (PoC)：TypeScript on Node.js —— 目标适配器生态 (Copilot SDK、MCP、OpenCode) 原生基于 Node/TypeScript，因此 TS/Node 提供一流的适配器，无需跨语言桥接
- V2 (生产)：Go —— 因其静态单一二进制输出而被预选用于生产分发；待生产范围开启后由后续 ADR 确认
- Python：已评估但未选择 —— 脚本编写速度的优势在 PoC 中无法胜过适配器集成成本

ADR 记录了完整的决策过程：问题、决策、状态、备选方案对比矩阵 (3 种语言 × 5 项标准)、论证、影响，以及可追溯到源故事和受影响工作制品的链接。这相当于将 DDD 的知识消化法应用到了架构决策中。

管线集成

utCodeAgentCLI 设计用于融入开发管线：

开发者目标 → utCodeAgentCLI

- 读取 methodPrompts/ (方法约束)
- 读取 slashCommands/ (执行单元)
- 规划：将目标分解为命令序列
- 执行：通过模型运行时调用命令
- 追踪：收集结构化执行日志
- 反思：分析结果、识别模式
- 输出：已验证的测试 + 追踪日志 + 对各层的反馈

specCodeAgentCLI 架构

specCodeAgentCLI 编排更大范围的生命周期。它基于 Px-SpecFlow，并在单元测试阶段复用 utCodeAgentCLI。

层次契约

关注点	所有者
SpecCoding 生命周期规则	Px-SpecFlow（位于 slashCommands/flows）
生命周期编排与排序	specCodeAgentCLI
单元测试设计与实现	utCodeAgentCLI（作为子 agent 调用）
CaTDD 方法语义	methodPrompts
可移植的命令执行单元	slashCommands

编排流程

```

1. 接收：导入 Issue/功能请求
2. 分析：转化为用户故事 → todoUS/
3. 开启：移至 doingUS/
4. 澄清意图：对齐开发者和 agent 的意图
5. 规划：分类工作方向，创建任务工作制品
6. 路由：
   ┌ 需求导向 → updateUserStory → reviewUserStory → commit/close
   ┌ 设计导向 → takeArchDesign → review → takeDetailDesign → review
   └ 实现导向 → designUnitTests → implUnitTests → implProductCodes
                                     ↓
                               reviewProductCodes
                                     ↓
                               commitWorks → closeUserStory

```

在每一步，specCodeAgentCLI 调用相应的 SPEC 命令，并在开发者（拥有产品意图）、SPEC 命令（拥有生命周期规则）和 utCodeAgentCLI（拥有测试执行）之间传递控制权。

子 Agent 模式

specCodeAgentCLI 将 utCodeAgentCLI 作为子 agent 对待：

```

specCodeAgentCLI: "故事 US-5 已就绪，可以开始实现。设计和实现 P0 测试。"
    ↓
utCodeAgentCLI: 读取 methodPrompts，读取测试文件，规划，执行，收集追踪记录
    ↓
utCodeAgentCLI: 返回："已设计 3 个 Typical、2 个 Edge、1 个 Misuse 测试。已实现 3 个 GREEN。剩余 3 个 ○ TODO。"
    ↓
specCodeAgentCLI: 更新故事状态。路由至 SPEC_reviewProductCodes。

```

这种分离将单元测试逻辑排除在生命周期编排器之外，也将生命周期逻辑排除在单元测试 agent 之外。

异步代码 Agent workflow

代码 agent 异步运行——你定义一个目标，调用 agent，并在完成后接收结果。这支持以下几种模式：

发射后不管

```

开发者: utCodeAgentCLI --goal "为 EventQueue 设计所有 P0 骨架"
Agent:   [自主工作数分钟]
Agent:   返回："P0 骨架已完成。4 个类别、8 个 US、14 条 AC、22 个 TC。可进行审查。"

```

开发者定义目标后去处理其他工作，Agent 完成后返回结果。

并行 Agent 会话

```
会话 A：utCodeAgentCLI --goal "EventQueue 的 P0 测试"
会话 B：utCodeAgentCLI --goal "CommandExecutor 的 P0 测试"
会话 C：specCodeAgentCLI --goal "分析待处理 Issue 转化为故事"
```

多个 agent 并发运行，每个聚焦于不同的范围。开发者批量审查结果。

管线串联

```
specCodeAgentCLI (分析 Issue → todoUS/ 中的故事)
  ↓ (触发条件：故事移至 doingUS)
specCodeAgentCLI (产出详细设计 → AC 设计完成)
  ↓ (触发条件：设计审查 PASS)
utCodeAgentCLI (设计 P0 测试 → 实现 → 验证)
  ↓ (触发条件：所有 P0 测试 GREEN)
specCodeAgentCLI (审查代码 → 提交 → 关闭 → doneUS)
```

每次 agent 调用触发管线中的下一步。开发者设置一次管线，然后监控检查点。

Agent 与开发者检查点的关系

代码 agent 是自主的，但并非无监督。CaTDD 工作流内嵌了检查点，开发者必须在 agent 继续之前批准：

检查点	Agent 状态	开发者动作
理解之后（阶段 1）	"是否继续进入测试设计？"	审查理解摘要。确认或修正。
设计之后（阶段 2）	"是否继续进入实现阶段？"	审查 US/AC/TC 设计。批准或要求修改。
关卡 P0 之后	"P0 完成。是否继续 P1？"	审查 P0 结果。决定是否需要 P1。
关卡 P1 之后	"架构已验证。是否继续 P2？"	审查架构测试结果。决定是否需要 P2。
实现之后	"测试完成。是否提交？"	审查最终报告。提交或要求变更。

Agent 遵守检查点契约：在每个关卡处停止并等待开发者输入。它不会自行推进。这保留了开发者对质量决策的掌控。

Agent 技能 vs 代码 Agent

有必要区分两个相关但不同的概念：

概念	是什么	位于何处
agentSkills	面向通用 CodeAgent（Copilot、Cline）的可复用能力包	agentSkills/ 一技能定义 + 生成的包
codeAgents	内建方法论知识的 CaTDD 原生 CLI agent	codeAgents/ 一架构 + 未来实现

agentSkills 的含义是："这是一个技能包。任何 CodeAgent 都可以加载并遵循 CaTDD。"

codeAgents 的含义是："我们是 CaTDD 原生 agent。我们了解方法，自主规划，并反馈学习成果。"

它们面向不同的受众：

- agentSkills 面向使用 Copilot/Cline/Continue 的开发者，希望其现有 agent 遵循 CaTDD。
- codeAgents 面向 CaTDD 项目本身——构建不依赖任何第三方 agent 的一流自动化。

Copilot 用户加载 `comment-alive-test-driven-development` 技能包。CaTDD 原生用户直接调用 `utCodeAgentCLI`。两者遵循相同的方法，产出相同的结构化输出。区别在于执行引擎。

代码 Agent 的未来

当前仓库记录了两个 agent 的预期层次契约。可运行实现在设计和开发中。架构包括：

- 基于 TypeScript 的 V1 (PoC) 用于 `utCodeAgentCLI` —— 快速迭代 AgentSDK 集成
- 基于 Go 的 V2 (生产) 用于 `utCodeAgentCLI` —— 编译后二进制文件，零运行时依赖
- SpecFlow 原生执行用于 `specCodeAgentCLI` —— 内建可追溯性的生命周期编排

核心洞察：代码 agent 不是方法论的事后补充，而是 CaTDD 演进路径的终点：

```
手动执行 (methodPrompts)
  ↓
命令化执行 (slashCommands)
  ↓
Agent 驱动执行 (codeAgents)
  ↓
打包可复用能力 (agentSkills)
```

每一层自动化了上一层更多的手动工作。代码 agent 是手动变为自主的转折点。

从自动化到集成

代码 agent 将 CaTDD 工作流自动化。但自动化建立在永恒的软件工程准则之上——即软件工程知识宝典。下一章 `applyClassicSWE` 将展示如何通过 CaTDD 对 TDD、BDD 和 DDD 三大流派的综合，在 LLM 时代充分释放这三门学科的完整表达能力。

Agent 生成测试。管线运行测试。团队审查测试。CaTDD 不会取代你的工程文化——它用结构化、可追溯、LLM 可读的验证设计来强化它。

applyClassicSWE

devBooks/ActionZH/05_applyClassicSWE.md

软件工程知识手册

软件工程不是时髦框架的集合，而是持久的、历经沧桑的知识体系——那些历经数十年仍被证明是构建复杂系统之真理的原则、实践与模式。软件工程知识手册包括：

学科	核心问题	标志性实践
TDD — 测试驱动开发	如何证明代码能正常工作？	先写失败测试，实现使其通过，重构
BDD — 行为驱动开发	如何将需求与代码连接起来？	GIVEN/WHEN/THEN 规格说明，可执行规格
DDD — 领域驱动设计	如何对复杂业务领域建模？	统一语言 (Ubiquitous Language)、限界上下文 (Bounded Context)、聚合 (Aggregate)、实体 (Entity)

这三门学科构成一个三角：TDD 验证行为，BDD 规格化行为，DDD 建模行为。三者共同回答了软件的全生命周期：构建什么（BDD）、如何建模（DDD）、以及如何证明它能正常工作（TDD）。

LLM 时代改变了一切——也什么都没改变

大语言模型已经改变了我们编写代码的方式。但它们并没有改变什么构成好的软件：

- 正确性依然重要——LLM 能够生成看起来合理但暗藏错误的代码
- 可追溯性依然重要——谁提出了这个功能需求？我们如何知道它已完成？
- 可维护性依然重要——只有 LLM 能读懂的代码是不可维护的
- 领域理解依然重要——除非你告诉 LLM，否则它不知道你的业务规则

LLM 是代码生成的力量倍增器。但如果没有结构，将混乱放大只会产生更多的混乱。经典的 SWE 学科——TDD、BDD、DDD——提供了防止 LLM 驱动开发沦为 LLM 驱动混乱所需的结构。

CoTDD 恰恰诞生于这个交叉点：它将软件工程知识手册转化为 LLM 可读的、注释鲜活的验证设计。它不取代 TDD、BDD 或 DDD，而是为 LLM 时代将它们融合为一体。

LLM 时代的 TDD

经典 TDD：基石

测试驱动开发由 Kent Beck 正式提出，有三条定律：

1. 在编写生产代码之前，先写一个失败的测试

- 2. 只写刚好足够失败的测试
- 3. 只写刚好足够通过的生产代码

Red→Green→Refactor 循环是 TDD 的心跳：

RED: 编写测试 → 测试失败（功能尚不存在）
GREEN: 编写最小化的生产代码 → 测试通过
REFACTOR: 在测试保持绿色的前提下，改进代码和测试结构

TDD 不是关于测试的，而是关于设计的。测试优先的纪律迫使你在实现之前思考接口、行为和契约。测试是你 API 的第一个客户。

TDD 在 LLM 之前的结构性局限

经典 TDD 有两个结构性缺口：

- 1. 没有设计制品：测试即是设计。但测试展示的是要验证什么，而非验证为何重要。业务意图——谁要求这个功能，它提供什么价值——隐含在开发者的脑海中，而不是在测试文件中显式地表达出来。
- 2. 手动工作量的上限：逐个编写测试是严谨的，但也是缓慢的。每个测试都必须由人工构思、编写、调试和验证。可能的测试场景总数总是超过可用于编写它们的时间。

CaTDD：针对 LLM 时代增强的 TDD

CaTDD 通过将设计意图和 LLM 可读性嵌入 TDD 循环来解决这两个缺口：

经典 TDD	CaTDD 增强
测试即是设计	注释骨架即设计——US/AC/TC 结构使业务意图显式化
测试什么内容是隐式的	优先级框架（P0→P3）使覆盖策略显式化
RED→GREEN→REFACTOR	○TODO→●RED→●GREEN，各优先级等级之间设有质量关卡
逐个手动编写测试	LLM 读取设计骨架，生成测试代码，实现最小化生产代码
重构是临时进行的	TC 逐条审查，因审查关卡失败触发重构
测试是代码制品	测试是鲜活的设计文档——注释随代码演进

LLM 并不只是更快地编写测试。它编写的是可追溯到设计意图的测试，因为设计意图被嵌入在 LLM 读取的同一个文件中。

CaTDD 的 TDD 循环（配合 LLM）

- 1. DESIGN: 编写注释骨架（US/AC/TC）—这就是设计
- 2. TC SELECTION: LLM 读取骨架，按优先级选择下一个 TC
- 3. RED: LLM 为 TC 生成测试代码 → 标记 ○→● → 测试失败
- 4. GREEN: LLM 生成最小化生产代码 → 测试通过 → 标记 ●→●
- 5. REVIEW: LLM 对照设计契约审查实现
- 6. REPEAT: 推进到下一个 TC
- 7. GATE: 在 P0/P1/P2 各关卡处停止，验证标准，请求开发者审批

步骤 3-5 在紧凑的 LLM 循环中进行。步骤 1、2、6、7 是人类判断发挥作用的节点。LLM 处理机械的 TDD 工作，开发者处理设计判断。

为什么 TDD 在 LLM 时代更重要

你可能会想："如果 LLM 能生成代码，为什么还要费心用 TDD？"答案是：

- LLM 生成看起来合理的代码比生成正确的代码更快。没有失败的测试，LLM 就没有反馈。没有通过的测试，你就没有信心。
- LLM 跨对话丢失上下文。测试文件就是上下文。CaTDD 注释骨架是代码应该做什么的持久记录，可被下一次 LLM 会话读取。
- LLM 是乐观的生成器。它们假设自己生成的代码是正确的。TDD 强加了验证的纪律。每个 GREEN 测试都是 LLM 无法幻觉越过的检查点。

TDD 在 LLM 时代并非不那么重要，而是**更加重要**。它是防止 LLM 生成代码漂入"看似合理但错误"领域的接地线。

LLM 时代的 BDD

经典 BDD：从需求到可执行规格

行为驱动开发由 Dan North 开创，弥合了业务利益相关者与开发者之间的鸿沟：

业务语言：	"作为客户，我想要取款，以便购买物品"
BDD 规格说明：	Scenario: 从账户取款
	Given 账户余额为 \$100
	When 客户取款 \$20
	Then 余额为 \$80
自动化测试：	步骤定义将 Given/When/Then 映射到测试代码

BDD 的核心洞见是：规格说明应该是可执行的。GIVEN/WHEN/THEN 场景既是业务需求，也是自动化测试。当测试通过时，需求即被满足。当测试失败时，需求即被破坏。

BDD 在 LLM 之前的结构性局限

经典 BDD 需要维护三个由人工编写的制品：

1. Feature 文件（Gherkin `.feature` 文件）——由产品负责人或业务分析师编写
2. 步骤定义（将 Gherkin 步骤映射到测试操作的代码）——由开发者编写
3. 生产代码（实现）——由开发者编写

跨越两种语言（Gherkin + 代码）的三个制品产生了维护摩擦。Feature 文件逐渐过期，步骤定义变得脆弱。"可执行规格"的愿景常常变成"曾经可执行的规格说明"。

CaTDD：嵌入测试文件的 BDD

CaTDD 将 BDD 以结构化注释的形式直接嵌入到测试文件中，消除了 Gherkin 与代码之间的间隙：

经典 BDD:

Feature: CashWithdrawal.feature

Scenario: Withdraw from account

Given the account has \$100

When the customer withdraws \$20

Then the balance is \$80

步骤定义: CashWithdrawal.java

@Given("account has \$100")

public void accountHas100() { ... }

测试运行器: Cucumber/JBehave

CaTDD BDD:

UT_Account.cxx:

// US-1: As a customer,

// I want to withdraw cash

// So that I can buy things.

//

// AC-1: GIVEN account has \$100,

// WHEN customer withdraws \$20,

// THEN balance is \$80.

//

// TC-1: verifyWithdraw_bySufficientBalance

// _expectReducedBalance

TEST(Account, verifyWithdraw_

bySufficientBalance_

expectReducedBalance) { ... }

CaTDD 的 US/AC/TC 链就是 BDD 规格，但它以注释的形式存在于测试文件中，而不是单独的 `.feature` 文件中。步骤定义和测试实现是同一个制品——TEST 函数。规格说明与验证之间的间隙不复存在。

BDD 作为 CaTDD 的验收标准契约

每个 CaTDD 验收标准都遵循 BDD 格式：

```
AC-n: GIVEN [前置条件与上下文],
      WHEN [特定动作或事件],
      THEN [预期的可观察结果],
      AND [如有额外的结果].
```

这并非巧合——它是 BDD 的 GIVEN/WHEN/THEN 格式作为 CaTDD 中规范的可测试条件。创建一个鲜活的文档调和检查清单来验证 BDD 一致性。

BDD 调和检查清单

建立自动化的调和测试，断言源代码、BDD 规格说明与外部文档之间的一致性：

- 强制每个 CaTDD AC 能映射到一个或多个 TC 实现（无孤立的验收标准，无未记录的测试用例）
- 验证 AC 描述与步骤特定的实现细节相区分——一个 TC 应验证一个 AC，而不是重复其他 AC 的步骤
- 生成清晰的 Markdown 报告，总结哪些 AC 已测试、哪些已规划、哪些已阻塞

为什么 BDD 在 LLM 时代更重要

LLM 是模式匹配器，而非业务分析师。它们能令人信服地生成 GIVEN/WHEN/THEN 结构，但无法判断 THEN 子句是否反映了真实的业务规则。配合 CaTDD 的 BDD 提供了护栏：

- 开发者编写 AC——以 GIVEN/WHEN/THEN 表达的业务规则
- LLM 生成 TC——验证 AC 的具体测试
- 开发者审查 TC——这个测试是否真正验证了 AC 所陈述的内容？
- LLM 实现测试代码——4 阶段实现
- 测试通过——AC 被满足

开发者拥有业务规则（AC），LLM 拥有验证机制（TC 与代码）。这种分工是为 LLM 时代优化的 BDD。

LLM 时代的 DDD

经典 DDD：对复杂领域建模

领域驱动设计由 Eric Evans 提出，是一种在软件中对复杂业务领域进行建模的方法论。其核心概念包括：

统一语言 (Ubiquitous Language)——领域专家与开发者之间的共享语言，在代码、对话和文档中统一使用。如果业务方说“保单”，代码中就有 `Policy` 类，而不是 `InsuranceMathHelper`。

限界上下文 (Bounded Context)——模型在其中适用的一个边界。在计费系统中，“Account”指支付账户，在 CRM 系统中，“Account”指客户账户。每个限界上下文有其自己的模型。

战略设计 (Strategic Design)——限界上下文之间的关系：核心域 (Core Domain)（你的竞争优势）、支撑子域 (Supporting Subdomains)（必要但无差异化）、通用子域 (Generic Subdomains)（购买，不自行构建）。

战术设计 (Tactical Design)——限界上下文内的构建块：实体 (Entities)（身份重于属性）、值对象 (Value Objects)（属性重于身份）、聚合 (Aggregates)（一致性边界）、领域事件 (Domain Events)（某事发生了）、仓储 (Repositories)（获取聚合）、服务 (Services)（无状态操作）。

DDD 在 LLM 之前的结构性局限

DDD 需要深厚的领域知识，这些知识难以传递给新团队成员：

- **统一语言是脆弱的**——一个开发者使用 `Customer`，另一个使用 `Client`，第三个对同一概念使用 `Account`。模型变得支离破碎。
- **限界上下文是隐式的**——它们存在于架构师的脑海和白板绘图上，而不是代码中或 LLM 的训练数据中。
- **模型演进是缓慢的**——重构领域模型需要更新文档、测试、生产代码和团队理解——全部手动进行。

CaTDD：作为 LLM 可读领域模型的 DDD

CaTDD 通过以下几种机制将 DDD 概念带入 LLM 时代：

1. OVERVIEW 部分作为限界上下文声明

每个 CaTDD 测试文件都以一个声明其限界上下文的 OVERVIEW 开头：

```
/**
 * [WHAT] This file verifies event queue behavior
 * [WHERE] in the IOC Event Subsystem module
 * [WHY] to ensure reliable asynchronous event delivery
 *
 * SCOPE:
 * - In scope: Event posting, queue capacity, consumer callback
 * - Out of scope: Event persistence (see UT_EventPersistence.cxx)
 *
 * KEY CONCEPTS:
 * - Conet vs Conles: Connection-oriented vs connection-less event modes
 * - EvtDescQueue: Fixed-capacity event descriptor queue
 * - CbExecCmd_F: Callback function for immediate command processing
 */
```

这个 OVERVIEW 是 DDD 的限界上下文，以 LLM 可读的注释形式呈现。它告诉 LLM：“在这个测试文件中，这些是领域概念。不要跨越到其他限界上下文。”

2. US/AC/TC 作为可执行领域规格

CaTDD 中的每个 User Story 映射到 DDD 的战略设计：

DDD 战略设计	CaTDD US/AC/TC
核心域 (Core Domain) 是什么让你独特？	→ P0 Functional (业务关键测试) Typical, Edge, Misuse, Fault
支撑子域 (Supporting Subdomain) → 必要但非竞争优势	P1 Design (架构测试) State, Capability, Interaction, Concurrency
通用子域 (Generic Subdomain) → 购买，不自行构建	P2 Quality (基础设施测试) Performance, Robust, Compatibility, Configuration, Diagnosis, Security
文档/示例 (Documentation/Examples)	→ P3 Addons (教程和演示)

优先级框架不止是测试排序——它是一种领域分类。P0 测试验证你的核心域。P1 测试验证你的架构。P2 测试检查你的质量属性。该框架告诉 LLM：“把精力集中在使该领域独特的部分上，而非优化基础设施。”

3. 设计骨架作为统一语言

CaTDD 设计骨架在测试文件中建立了统一语言：

```
// @[Class]: P0 Functional / ValidFunc
// @[Category]: Typical
// @[Intent]: Prove the core event posting and delivery workflow
// @[UseWhen]: Producer and consumer are valid, queue has capacity
// @[AvoidWhen]: Queue is full, consumer is blocking, or producer is invalid
// @[US]: US-1 (event posting), US-2 (event delivery)
// @[AC]: AC-1 through AC-4
// @[TC]: TC-1 verifyEventPost_byAvailableCapacity_expectEventQueued
//          TC-2 verifyEventDelivery_byCallbackConsumer_expectEventProcessed
```

该骨架一致地使用了领域术语：“producer”、“consumer”、“queue”、“capacity”、“blocking”。这些不是通用的测试术语——它们是该领域的统一语言。LLM 读取骨架后，在生成任何代码之前就理解了领域模型。

4. SpecFlow 制品作为领域知识

Px-SpecFlow 的生命周期制品将领域知识保存在版本控制的文件中：

.catdd/spec/projectContext.md	→ 领域愿景、限界上下文、约定
README_ArchDesign.md	→ 战略设计、上下文映射
README_DetailDesign.md	→ 战术设计、聚合边界
README_VerifyDesign.md	→ 领域验证拓扑

这些制品是 DDD 的知识结晶输出，但以 LLM 可读的 Markdown 形式存在，并且能经受代码变更的考验。

DDD 调和检查清单

使用以下检查点验证 CaTDD 制品中的 DDD 一致性：

- 扫描测试文件中的领域术语一致性——同一概念在所有测试文件中是否以相同方式命名？
- 验证限界上下文边界——`UT_EventQueue.cxx` 是否仅引用 `EventQueue` 领域概念，还是泄漏到 `CommandExecutor` 概念中？
- 检查 OVERVIEW 部分是否足够清晰地声明其限界上下文，使 LLM 能够理解
- 对每个聚合根，验证是否存在相应的测试文件，具有 US/AC/TC 覆盖

为什么 DDD 在 LLM 时代更重要

LLM 是领域无关的。它们懂得语法但不懂语义。它们能为任何领域生成代码，但对任何领域都不理解。

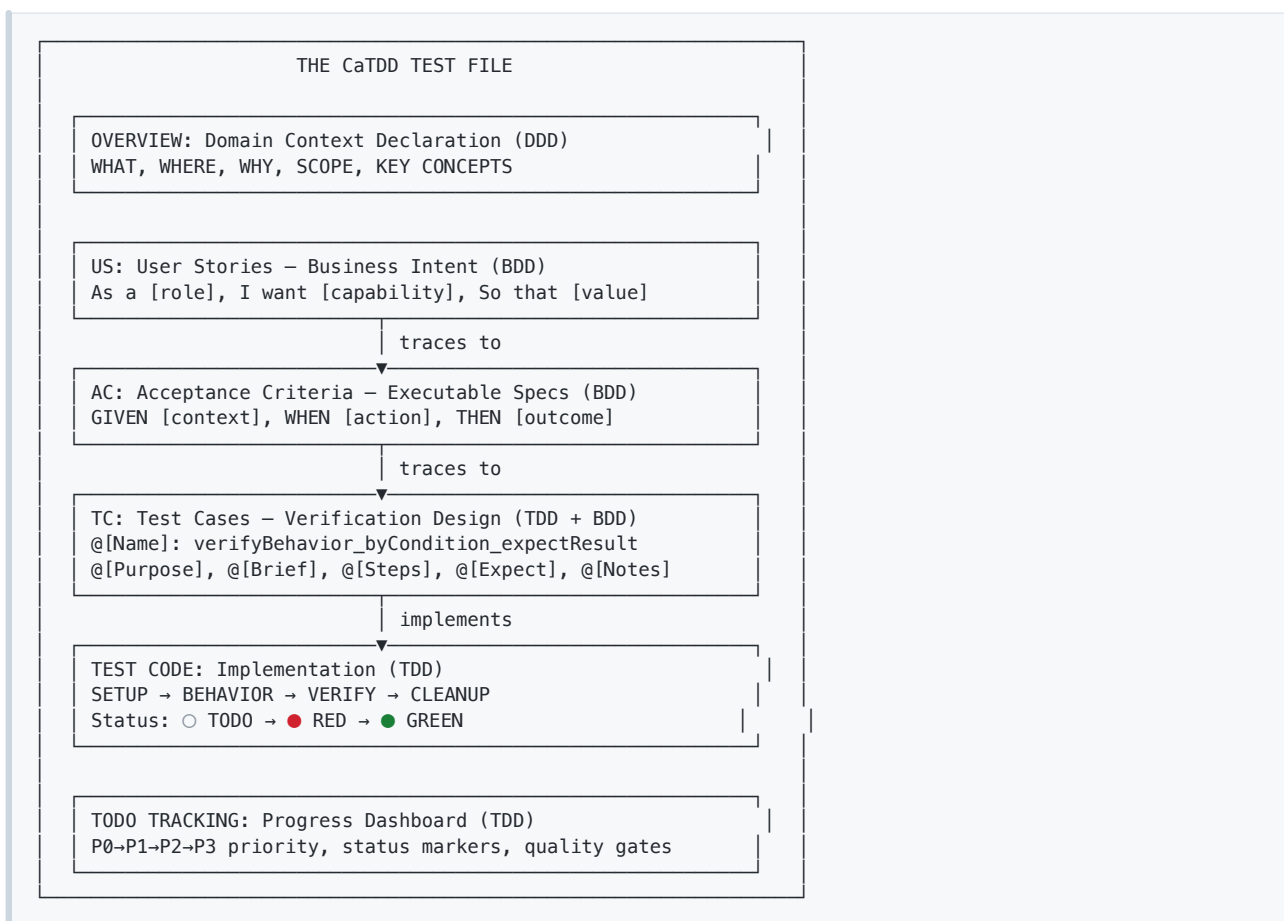
DDD 填补了这一空白：

- 统一语言为 LLM 提供一致的术语——LLM 读取测试文件时，看到的是 "event producer"、"event consumer"、"EvtDescQueue" 这些一致使用的术语。它从测试注释中学习领域词汇。
- 限界上下文防止 LLM 混合模型——Billing.Account 的测试文件使用计费术语，CRM.Account 的测试文件使用 CRM 术语。LLM 不会混淆它们，因为 OVERVIEW 声明了它所处的上下文。
- 战略设计告诉 LLM 要优先关注什么——优先级框架（P0 核心域，P1 支撑，P2 通用）告诉 LLM 在有限的上下文窗口中把精力集中在哪里。

没有 DDD，LLM 会将每个测试文件视作一个扁平的、可互换的代码制品。通过将 DDD 嵌入 CaTDD 注释，LLM 能理解领域架构。

CaTDD：TDD + BDD + DDD 的融合

CaTDD 不是堆叠在三个学科之上的第四个方法论，而是一种融合——TDD、BDD 和 DDD 在一个单一、LLM 可读的制品（测试文件）中共存协作的方式：



一个单一的测试文件包含：

- DDD 的限界上下文（OVERVIEW 部分）
- BDD 的可执行规格（US/AC 链，使用 GIVEN/WHEN/THEN）
- TDD 的验证循环（TC 实现，使用 RED→GREEN 标记）

- DDD 的战略优先级排序 (P0→P3 框架映射到核心/支撑/通用)

这就是融合。一个文件。三门经典学科。全程 LLM 可读。

LLM 作为知识手册阅读器

软件工程知识手册教导我们：

原则	含义	CaTDD 如何应用
先设计后编码	在构建之前理解要构建什么	注释骨架即设计
先验证后发布	交付前证明正确性	RED→GREEN 循环 + 质量关卡
说领域语言	在代码中使用业务术语	OVERVIEW + US/AC 使用领域词汇
限定模型边界	每个上下文一个模型，不要混用	每个聚合/限界上下文一个测试文件
从外向内测试	验证行为，而非实现	GIVEN/WHEN/THEN 关注可观察结果
小的、可逆的变更	小步提交，轻松回滚	每次一个 TC，RED→GREEN 提交
知识在代码中	不要让知识只存在于脑海	注释鲜活的设计保留决策

LLM 时代并没有否定这些原则，反而更加迫切地需要它们——因为没有这些原则，LLM 生成的代码就没有护栏。

LLM 通过 CaTDD 阅读知识手册

当你将一个 CaTDD 测试文件分享给 LLM 时，你分享的是以注释编码的知识手册：

LLM reads:	OVERVIEW section	→ 学习限界上下文和领域概念
LLM reads:	US section	→ 学习业务价值和参与者角色
LLM reads:	AC section	→ 学习可执行规格说明 (BDD)
LLM reads:	TC section	→ 学习要验证什么以及如何验证
LLM reads:	TODO tracking section	→ 学习当前进度和下一步行动
LLM reads:	TEST CODE section	→ 学习实现模式
LLM reads:	@[Category] tags	→ 学习测试优先级和风险等级

LLM 无需了解 DDD 术语即可从 DDD 中获益。它读取的测试文件以通俗易懂的方式声明了其限界上下文（OVERVIEW），使用一致的领域术语（US/AC 中的统一语言），并遵循优先级规则（P0→P3 中的战略设计）。方法论被嵌入在制品中。

BDD → CaTDD：从 Feature 文件到注释链

经典 BDD 使用独立于测试代码的 Gherkin `.feature` 文件：

```

Feature: Cash Withdrawal
  As a customer
  I want to withdraw cash from my account
  So that I can make purchases

  Scenario: Successful withdrawal
    Given my account has a balance of $100
    When I withdraw $20
    Then my balance should be $80
    And the ATM should dispense $20

  Scenario: Insufficient funds
    Given my account has a balance of $50
    When I withdraw $100
    Then the withdrawal should be rejected
    And my balance should remain $50

```

CaTDD 将其转换为测试文件中的注释链：

```

// US-1: As a customer,
//       I want to withdraw cash
//       So that I can make purchases.
//
// AC-1: GIVEN account has balance $100,
//       WHEN customer withdraws $20,
//       THEN balance is $80,
//       AND ATM dispenses $20.
//
// AC-2: GIVEN account has balance $50,
//       WHEN customer withdraws $100,
//       THEN withdrawal is rejected,
//       AND balance remains $50.

// [@AC-1,US-1] P0 Functional / Typical
// TC-1:
//   @[Name]: verifyWithdraw_bySufficientBalance_expectReducedBalance
//   @[Purpose]: Ensure valid withdrawals are processed correctly
//   @[Brief]: Set up account with $100, withdraw $20, verify balance $80
//   @[Expect]: Balance is $80, dispense event is triggered

TEST(AccountWithdraw, verifyWithdraw_bySufficientBalance_expectReducedBalance) {
  //===SETUP===
  Account acc(100);

  //===BEHAVIOR===
  auto result = acc.withdraw(20);

  //===VERIFY===
  ASSERT_TRUE(result.success);
  ASSERT_EQ(80, acc.balance());
  ASSERT_TRUE(result.dispensed);
}

```

这种转换保留了 BDD 的结构，但消除了规格说明与测试之间的分离。GIVEN/WHEN/THEN 存在于 AC 注释中。场景实现存在于其正下方的 TEST 函数中。LLM 在读取此文件时，在一个连续的上下文中看到了从业务需求到可执行验证的完整链条。

DDD → CaTDD：从领域模型到注释骨架

经典 DDD 产出一个以类图、上下文映射和设计文档形式表达的领域模型：

```

DDD 制品：
- Bounded Context: Account Management
- Aggregate Root: Account
- Entity: Transaction
- Value Object: Money
- Domain Service: TransferService
- Ubiquitous Language: balance, withdraw, deposit, transfer, overdraft

```


CaTDD 将此领域模型嵌入到测试文件的 OVERVIEW 和设计骨架中：

```
//=====
//=====>BEGIN OF OVERVIEW=====
/**
 * [WHAT] This file verifies Account aggregate behavior
 * [WHERE] in the Account Management bounded context
 * [WHY] to ensure correct balance operations and transfer integrity
 *
 * SCOPE:
 * - In scope: Account withdrawal, deposit, transfer logic
 * - Out of scope: User authentication (see UT_Authentication.cxx)
 * - Out of scope: Notification delivery (see UT_Notification.cxx)
 *
 * KEY CONCEPTS:
 * - Account (Aggregate Root): Manages balance and transaction history
 * - Money (Value Object): Immutable amount + currency pair
 * - Transaction (Entity): Recorded entry in account history
 * - TransferService (Domain Service): Coordinates two-account transfer
 */
//=====>END OF OVERVIEW=====
```

LLM 读取此 OVERVIEW 并理解到：

- 该测试文件覆盖一个限界上下文（Account Management）
- 聚合根是 Account——测试应针对 Account 行为
- Money 是值对象——测试应验证值相等性，而非标识符
- Transaction 是实体——测试应验证它被正确记录
- 该文件不测试身份认证或通知（超出范围）

这是通过结构化注释传递给 LLM 的 DDD 知识。

P0→P3 框架作为战略 DDD

CaTDD 的优先级框架直接映射到 DDD 的战略设计：

DDD 战略分类	CaTDD 优先级映射
核心域 (Core Domain) (你的竞争优势) — 创造利润的内容 — 对你业务独有的内容 — 在这里投入最多精力	P0 Functional (Typical, Edge, Misuse, Fault) — 业务关键 workflow — 领域特定的行为 — 在任何其他测试之前完成
支撑子域 (Supporting Subdomain) (必要但非独有) — 支撑核心域的内容 — 可以外包的内容 — 投入足够保持它运行的精力	P1 Design (State, Capability, Interaction, Concurrency) — 架构验证 — 生命周期与 FSM — 在 P0 之后完成
通用子域 (Generic Subdomain) (购买，不自行构建) — 所有业务都通用的内容 — 使用现有解决方案 — 最小化定制投入	P2 Quality (Performance, Robust, Compatibility, Configuration, Diagnosis, Security) — 非功能性质量 — 平台兼容性 — 在 P1 之后完成
文档/教育 (Documentation / Education) (演示、示例、教程)	P3 Addons (Demo/Example)

CaTDD CodeAgent 读取此映射后理解到：

- "P0 测试 = 这是使该领域有价值的内容——大力投入"

- "P1 测试 = 这是支撑核心的内容——充分测试架构"
- "P2 测试 = 这是基础设施——验证质量，不要过度测试"

这防止了 LLM 常见的一种反模式：将所有测试同等对待。LLM 学会基于领域重要性来排优先级，而不仅基于代码覆盖率指标。

LLM 时代的鲜活文档

知识手册教导我们，文档必须随代码演进而演进。CaTDD 通过注释鲜活的设计将这一原则具体化：

鲜活词汇表

CaTDD 测试文件通过注释中一致的领域术语建立起统一语言：

- 仅扫描目标限界上下文中的文件
 - 过滤掉测试工具、框架配置和样板代码
 - 测试文件名中的类名映射到领域术语（例如 `UT_Account.cxx` → Account 聚合）
 - AC 注释定义了术语行为："GIVEN account has balance \$100"——术语 "balance" 由此条件所定义
- LLM 通过阅读 OVERVIEW 部分和 AC 注释来构建其领域词汇表。无需单独的词汇表文件。

鲜活图表

SpecFlow 面向架构的 SPEC 文档充当鲜活图表：

- `README_ArchDesign.md` → 限界上下文映射，以 LLM 可读的文本形式表达
- `README_DetailDesign.md` → 聚合边界和类关系
- `README_VerifyDesign.md` → 测试拓扑和验证覆盖

这些是文本文档，而非图片。LLM 能够解析它们。但它们在人类需要可视化时也可以使用 Mermaid 或 PlantUML 渲染为图表。文本是真理之源——图表是一种展示视图。

CaTDD 中的 BDD 调和

经典 BDD 中的调和意味着验证 Feature 文件、步骤定义和生产代码之间的一致性。在 CaTDD 中，调和更简单，因为不存在间隙：

- Feature 文件 = 测试文件中的 AC 注释
- 步骤定义 = 同一文件中的 TC 注释 + TEST 函数
- 生产代码 = 在针对该 TC 的 GREEN 阶段中实现

调和检查变为：

- 每个 AC 至少有一个追溯到它的 TC
- 每个 TC 的断言实际验证了它声称要验证的 AC
- 每个生产代码的变更都是由一个 RED TC 所驱动的

这些检查可以通过解析注释标记来自动化——无需外部 BDD 工具。

应用知识手册：一个完整示例

让我们追踪一个特性经过 TDD、BDD 和 DDD，看看 CaTDD 如何应用它们：

1. BDD：表达需求

```
US-1: As a customer,
      I want to transfer money between my accounts
      So that I can manage my funds across accounts.

AC-1: GIVEN checking account has $500 and savings account has $200,
      WHEN customer transfers $100 from checking to savings,
      THEN checking balance is $400,
      AND savings balance is $300.

AC-2: GIVEN checking account has $100,
      WHEN customer transfers $200 from checking to savings,
      THEN transfer is rejected,
      AND checking balance remains $100,
      AND savings balance is unchanged.
```

2. DDD：在 OVERVIEW 中对领域建模

```
/**
 * [WHAT] This file verifies cross-account money transfer
 * [WHERE] in the Account Management bounded context
 *
 * KEY CONCEPTS:
 * - Transfer (Domain Operation): Moves Money between two Accounts
 * - Money (Value Object): amount + currency, immutable
 * - Account (Aggregate Root): owns balance and transaction history
 * - TransferLimit (Policy): maximum per-transfer amount
 */
```

3. TDD：设计测试用例

```
// P0 Functional / Typical - Core Domain
// TC-1: verifyTransfer_byBasicTransfer_expectUpdatedBalances
//   @[Purpose]: Verify the fundamental transfer operation
//   @[Brief]: Set up two accounts, transfer $100, verify both balances

// P0 Functional / Edge - Domain boundaries
// TC-2: verifyTransfer_byExceedingBalance_expectRejected
//   @[Purpose]: Verify overdraft protection
//   @[Brief]: Attempt to transfer more than available balance

// P0 Functional / Edge - Domain precision
// TC-3: verifyTransfer_byZeroAmount_expectNoOperation
//   @[Purpose]: Verify zero-amount transfer is a no-op
//   @[Brief]: Transfer $0, verify no balance change

// P0 Functional / Misuse - Domain misuse
// TC-4: verifyTransfer_bySameAccount_expectRejected
//   @[Purpose]: Prevent transfer to the same account
//   @[Brief]: Set source and destination to same account
```

4. LLM：实现 TDD 循环

LLM 读取 US/AC/TC 设计（BDD 规格说明 + DDD 领域模型）并实现：

```
TC-1: RED → 实现 TransferService.transfer($100) → GREEN
TC-2: RED → 实现 InsufficientFunds 检查 → GREEN
TC-3: RED → 实现零金额防护子句 → GREEN
TC-4: RED → 实现同账户检查 → GREEN
```

每个实现都被领域所限定：LLM 从 OVERVIEW 中知道 Transfer 是一个领域操作，Money 是一个值对象，Account 是一个聚合根。生成的代码使用了这些术语。

5. 审查：对照知识手册进行验证

- ✓ BDD: 每个 AC 有一个或多个 TC—可执行规格存在
- ✓ DDD: OVERVIEW 声明了限界上下文—领域模型显式化
- ✓ TDD: 所有 TC 遵循 RED→GREEN 循环—验证完整
- ✓ 融合: 变更可追溯: US → AC → TC → Code → Git commit

知识手册应用于实践

软件工程知识手册教导的是持久的、历久弥新的原则。CaTDD 是在 LLM 时代对它们的应用：

知识手册原则	CaTDD 实践
TDD: 测试优先，代码其次	注释骨架优先，LLM 生成 TC → RED → GREEN
BDD: 规格化行为，而非实现	US/AC/TC 链即是可执行规格
DDD: 说领域语言	OVERVIEW + US/AC 使用一致的领域术语
DDD: 限定模型边界	每个限界上下文/聚合根一个测试文件
TDD: 小的、已验证的步骤	每次一个 TC，○→●→● 标记
BDD: 将规格与测试放在一起	AC 注释与 TEST 函数在同一文件中
DDD: 战略优先级排序	P0(核心)→P1(支撑)→P2(通用)→P3(文档)
SWE: 知识在代码中	注释鲜活的设计将决策保留在制品中

LLM 时代并没有使 TDD、BDD 或 DDD 过时，而是使它们可操作化。CaTDD 就是在 LLM 硬件上运行 TDD、BDD 和 DDD 的操作系统。

将知识手册打包为 Agent Skills

知识手册不仅仅存在于人类编写的书籍中。CaTDD 通过两个可复用的 agent skills 将其操作化，任何 CodeAgent 都可以加载：

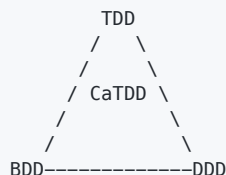
comment-alive-test-driven-development —— 将 CaTDD 打包为可复用的能力。当 Copilot 用户加载此 skill 并说 "use CaTDD" 时，agent 即拥有完整的知识手册：完整的 `CaTDD_methodPrompt.md`、`CaTDD_designAndImplTemplate.cxx` 模板以及 `slashCommands` 流程素材。该 skill 告诉 agent 谁使用它、创建什么（测试文件作为鲜活的设计文档）、何时应用它、输出放在何处、为什么重要（结构化注释为 LLM 提供生成正确代码所需的上下文）以及如何执行（4 个阶段：Scope → Design → Implement → Validate）。

user-story-centered-spec-coding —— 将 SpecCoding 生命周期打包为可复用的工作流。它告诉 agent 如何将工作从 `pendingNews` → `todoUS` → `doingUS` → `doneUS` 推进，其中 `abortUS` 保留不安全的活跃故事以供后续分析，CaTDD 作为默认测试方法，typical TDD 作为备选方案。该 skill 捆绑了完整的 Px-SpecFlow 生命周期：流程文档、所有 27 个 `SPEC_*` 命令文件、项目根 README SPEC 模板以及作为默认测试引擎的 CaTDD 方法引用。

两个 skills 均由 `agentSkills/makeSkill.sh` 从原创源文件生成。原创源文件是持久的资产；生成的包是

构建输出。这种分离保护了知识手册的完整性——方法在单一位置（methodPrompts）进行变更，并通过构建脚本传播到所有 skill 包。

结语：经典 SWE 的三角



TDD proves behavior.
BDD specifies behavior.
DDD models behavior.
CaTDD synthesizes all three into
one LLM-readable test file.

软件工程知识手册不是过往实践的博物馆，而是构建正确的、可维护的、领域一致的软件的运维手册——无论有没有 LLM。CaTDD 即是一种方法论，它阅读该手册并将其转化为注释鲜活的验证设计，LLM、开发者和 code agents 都能理解、执行和改进。

Comments is Verification Design. LLM Generates Code. Iterate Forward Together.

beyondXyzSpec

devBooks/ActionZH/06_beyondXyzSpec.md

规格驱动开发现状

LLM 时代已催生出三个主要的规格驱动开发开源方案。它们共享一个信念——规格先于代码 (spec before code)——但在核心理念和适用范围上存在根本差异。

	GitHub Spec Kit	OpenSpec	CaTDD Px-SpecFlow
核心理念	文档 (spec.md, plan.md)	变更 (openspec/changes/)	用户故事 (User Stories) (US 生命周期)
Star 数	111k	54k	—
CLI	Python (specify)	TypeScript/Node.js (openspec)	Markdown prompt 命令 (agent 无关)
Agent 集成	30+	25+	Copilot, Cline, Continue, utCodeAgentCLI
测试方法论	未嵌入	未嵌入	嵌入 — UT P0/P1/P2 配合 TDD

本章审视三者，并阐释 CaTDD 的双流架构——SPEC 生命周期 + UT 验证——为何超越了独立规格驱动工具所能提供的范畴。

GitHub Spec Kit — 以文档为中心的规格驱动开发

GitHub Spec Kit (111k stars, github/spec-kit) 是一个基于 Python 的工具包，围绕四个核心文档组织开发：

```
constitution.md → spec.md → plan.md → tasks.md → implement
```

架构

制品	位置	承载内容
Constitution	.specify/memory/constitution.md	项目原则、质量标准、治理规则
Spec	specs/001-feature/spec.md	WHAT & WHY — 用户故事、功能需求
Plan	specs/001-feature/plan.md	HOW — 技术栈、架构、实现决策
Tasks	specs/001-feature/tasks.md	可执行的任务清单，含依赖与 [P] 并行标记

命令流程

```

/speckit.constitution  - 创建项目治理原则
/speckit.specify       - 定义构建什么（需求、用户故事）
/speckit.clarify       - 澄清规格不足的地方（制定计划前必须执行）
/speckit.plan          - 定义如何构建（技术栈、架构）
/speckit.tasks         - 将计划分解为可操作任务及依赖
/speckit.analyze       - 跨制品一致性与覆盖分析
/speckit.checklist     - 生成质量检查清单（“英语的单元测试”）
/speckit.implement     - 执行所有任务以构建特性

```

核心能力

- 扩展与预设 (Extensions & Presets)：定制或扩展工作流。Extension 添加新命令（如 Jira 集成），Preset 覆盖模板（如合规格格式、领域术语）。`.specify/templates/overrides/` 中的项目本地覆盖具有最高优先级。
- 安装方式：`uv tool install specify-cli`，然后 `specify init <project> --integration copilot`
- 并行任务执行：标记 [P] 的任务并发运行，未标记 [P] 的任务遵守依赖顺序。

Spec Kit 止步于何处

Spec Kit 是一个从规格到任务的流水线。它告诉你构建什么以及按什么顺序，但除了通用检查清单之外，它不告诉你如何验证正确性。`/speckit.checklist` 命令生成质量验证条目，但没有嵌入的测试方法论，没有 US/AC/TC 可追溯链，没有 TDD 纪律，也没有验证状态跟踪。一旦运行 `/speckit.implement`，规格的任务即告完成——规格不会作为可验证的鲜活制品继续存在。

CaTDD 从 Spec Kit 采纳的内容

CaTDD 的 Px-SpecFlow 明确记录了从 Spec Kit 采纳的六项改进，每项都适配了 CaTDD 方法论：

Spec Kit 概念	CaTDD 采纳
Constitution 治理所有决策	<code>.catdd/spec/projectContext.md</code> 作为共享护栏—— <code>SPEC_initProjectContext</code> 在故事工作开始前记录稳定的原则、约束和团队约定
按优先级排序的、可独立测试的用户故事	<code>SPEC_analyzeIssue</code> 产出的故事包含参与者、价值、优先级、独立测试意图、验收场景、边缘情况、风险和开放问题——从分析阶段即为可测试性构建
设计前明确意图	<code>SPEC_clearStoryIntent</code> 在 <code>SPEC_openUserStory</code> 之后记录相互意图契约 (Mutual Intent Contract)：开发者意图、CodeAgent 意图、范围、非目标、成功信号、假设、开放问题
将 WHAT 与 HOW 分离	<code>SPEC_makePlan</code> 创建配对的 <code>*-TASKs.md</code> 制品，将工作分类为意图澄清、需求导向、设计导向或实现导向——技术选择落在项目根 <code>README* SPEC</code> 文档中
编码前澄清/分析	<code>SPEC_reviewArchDesign</code> 和 <code>SPEC_reviewDetailDesign</code> 作为设计质量关卡；失败的审查路由至 <code>SPEC_update*Design</code> ，然后再进行下游工作
显式的、支持并行的执行切片	US/AC/TC 切片，PO 优先排序，独立工作标记为可并行执行

CaTDD 采纳了 Spec Kit 的结构智慧，但将其嵌入到一个以用户故事为中心、配有集成验证引擎的生命周期中。

OpenSpec — 以变更为中心的规格驱动开发

OpenSpec (54k stars, [Fission-AI/OpenSpec](#)) 是一个基于 TypeScript 的工具，构建于不同的理念之上：流动而非僵化，迭代而非瀑布，简易而非复杂，为既有代码库 (brownfield) 而设计，非仅全新项目 (greenfield)。

架构

OpenSpec 围绕变更 (changes) 组织工作——`openspec/changes/` 中自包含的文件夹。每个变更都是一个关于构建或修改的提案：

```
openspec/changes/add-dark-mode/
├─ proposal.md  - 为什么做这个、变更的内容是什么
├─ specs/       - 需求和场景
├─ design.md    - 技术方案
└─ tasks.md     - 实现检查清单
```

命令流程

核心工作流 (3 个命令)：

```
/opsx:propose "add-dark-mode"  - 一步创建 proposal + specs + design + tasks
/opsx:apply                    - 实现所有任务
/opsx:archive                  - 移至归档，更新 specs
```

扩展工作流 (额外 6 个命令)：

```
/opsx:new          - 以引导上下文创建新变更
/opsx:continue     - 恢复活跃变更上的工作
/opsx:ff           - 快速完成简单任务
/opsx:verify       - 对照 specs 验证实现
/opsx:bulk-archive - 一次归档多个已完成的变更
/opsx:onboard      - 通过引导探索走查系统
```

核心能力

- 变更文件夹隔离：每个变更位于自己的文件夹中，特性之间无交叉污染。变更是自包含的且可独立归档。
- 仪表板 (Dashboard)：所有变更的可视化全景，显示状态和关系。
- 优先适配既有代码库 (Brownfield-first)：专为现有代码库设计，而不仅是全新项目。你为已存在的内容提出变更提案。
- 安装方式：`npm install -g @fission-ai/openspec`，然后 `openspec init`
- 自我对比："vs Spec Kit — 全面但重量级，僵化的阶段关卡。OpenSpec 更轻量，让你自由迭代。"

OpenSpec 止步于何处

OpenSpec 是一个变更管理系统，配有轻量级规格脚手架。每个变更都有 proposal、specs、design 和 tasks。但和 Spec Kit 一样，它在实现阶段止步——没有嵌入的测试方法论，没有 US/AC/TC 可追溯性，也没有验证生命周期。规格描述要变更什么，但不定义如何证明变更正确。变更被归档时即视为"完成"——没有推动闭环的 RED→GREEN 测试状态。

CaTDD Px-SpecFlow — 以用户故事为中心的规格编码

CaTDD Px-SpecFlow 既不是以文档为中心，也不是以变更为中心，而是以用户故事为中心的。用户故事是核心生命周期制品。每个 SPEC 命令都服务于故事从接收到已验证、已审查、已提交闭环的旅程。

故事生命周期

```

pendingNews/ → analyzedNews/ → todoUS/ → doingUS/ → doneUS/
(原始输入)   (输入存档)       (已分析)   (活跃中)     (已完成)
              ↘ suspendUS/ (暂停, 可恢复)
              ↘ abortUS/  (已保留)
  
```

七个生命周期目录，每个都在 `.catdd/spec/` 下的版本控制目录中：

状态	目录	含义
pendingNews	pendingNews/	等待分析的原始 issue、feature 或导入的用户故事
analyzedNews	analyzedNews/	分析后的原始输入，作为源追溯保留
todoUS	todoUS/	已分析的用户故事，准备好被开启工作
doingUS	doingUS/	活跃的用户故事，处于设计、测试或实现阶段
suspendUS	suspendUS/	暂停的活跃故事，附带持久的恢复引用（分支/worktree）被保留
abortUS	abortUS/	已中止的故事，保留以供后续重新分析或改进
doneUS	doneUS/	已完成的故事，通过审查、提交和 CI 之后

SPEC 命令家族

Px-SpecFlow 提供 34 个 SPEC 命令，组织为三个生命周期阶段：

阶段 A — 故事前：输入与分析

```

SPEC_initProjectContext → .catdd/spec/projectContext.md (共享章程)
SPEC_updateProjectContext → 更新项目上下文
SPEC_importIssue → 原始 issue → pendingNews/
SPEC_importFeature → 原始 feature → pendingNews/
SPEC_importUserStory → 结构化的 US → todoUS/ (跳过分析)
SPEC_analyzeIssue → pending 输入 → todoUS/ 中的用户故事 + 存档于 analyzedNews/
SPEC_analyzeFeature → pending 输入 → todoUS/ 中的用户故事 + 存档于 analyzedNews/
SPEC_analyzeAbortedUserStory → 对已中止故事进行选择性重新分析，供下一轮使用
SPEC_openUserStory → 将选定故事从 todoUS/ 移至 doingUS/ (工作开始)
  
```

阶段 B — 设计与规划

```

SPEC_clearStoryIntent → 对齐开发者与 CodeAgent 的意图 (相互意图契约)
SPEC_makePlan → 分类工作导向，创建 *-UserStory-Tasks.md，选择下一步命令
SPEC_updateUserStory → 更新模块 README_UserStory.md + README_UserGuide.md
SPEC_reviewUserStory → 在下游工作前把关需求质量
SPEC_whatsNextTask → 根据当前状态推荐唯一下一步命令
SPEC_takeArchDesign → 初始架构设计 (README_ArchDesign.md + 7 个其他文件)
SPEC_reviewArchDesign → 把关架构质量
SPEC_updateArchDesign → 后续架构修订
SPEC_takeDetailDesign → 初始详细设计 (README_DetailDesign.md + README_StateDesign.md)
SPEC_reviewDetailDesign → 把关详细设计质量
SPEC_updateDetailDesign → 后续详细设计修订
  
```

阶段 C — 实现与闭环

SPEC_designUnitTests	→ 进入 CaTDD 测试设计（路由至 P0/P1/P2 UT 流程）
SPEC_implUnitTests	→ 实现测试用例（通过 UT 命令进行 RED→GREEN）
SPEC_reviewImplUnitTests	→ 在实现产品代码前把关单元测试实现
SPEC_implProductCodes	→ 实现生产代码以通过测试
SPEC_reviewProductCodes	→ 审查实现质量
SPEC_refactUnitTests	→ 对单个 GREEN 单元测试进行无行为变更的清理
SPEC_suspendUserStory	→ 暂停活跃故事 → suspendUS/，附恢复引用
SPEC_resumeUserStory	→ 将暂停的故事移回 doingUS/ 并继续
SPEC_partialCloseUserStory	→ 关闭已接受范围，将拒绝范围移至 abortUS/
SPEC_abortUserStory	→ 中止活跃故事 → abortUS/（保留以供重新分析）
SPEC_commitWorks	→ 准备并提交已完成的工作
SPEC_closeUserStory	→ 将已审查、已提交的故事移至 doneUS/
SPEC_mergeWorks	→ 当仍需集成时，合并已关闭的故事分支
SPEC_patchOriginalCaTDD	→ 将已安装项目的 CaTDD 改进补丁回写上游

CaTDD 的独特之处

Px-SpecFlow 的创新超越了 Spec Kit 和 OpenSpec 所提供的范畴：

1. 以用户故事为中心，而非以文档为中心或以变更为中心。Spec Kit 围绕 spec.md/plan.md/tasks.md 这些文档展开。OpenSpec 围绕 changes/ 这些提案文件夹展开。CaTDD 围绕用户故事本身作为鲜活制品展开，经历七个生命周期目录。每个 SPEC 命令都服务于故事的旅程。故事从原始导入、经分析存档、分析待办、活跃进行中、可能暂停或中止、到最终闭环，拥有显式的存在状态。
2. 相互意图契约。Spec Kit 和 OpenSpec 都没有在开始设计前对齐人类与 LLM 意图的机制。CaTDD 的 SPEC_clearStoryIntent 记录一个契约：开发者认为故事是什么，CodeAgent 推断出什么，范围内工作，范围外工作，成功信号，假设和开放问题。意图未对齐，设计不开始。
3. SPEC_abortUserStory 一显式的失败保留。当活跃故事存在阻塞性的范围问题、无效假设或不应就地修补的质量问题时，CaTDD 将故事中止到 abortUS/ 中作为保留历史。中止生命周期反馈到 SPEC_analyzeAbortedUserStory 或 SPEC_importIssue，以便进行深思熟虑的重新分析。Spec Kit 和 OpenSpec 都没有显式的中止并保留路径。
4. SPEC_makePlan 一工作导向分类。在任何下游工作之前，SPEC_makePlan 将故事分类为四种导向之一：意图澄清、需求导向、设计导向或实现导向。它区分初始设计（SPEC_take*Design）与后续修订（SPEC_update*Design）。它创建配对的 *-UserStory-Tasks.md 制品，以 Markdown 复选框任务的形式展示所需的下一步步骤。
5. 项目根 README SPEC 文档。CaTDD 管理 13 种项目根 README SPEC 文档类型，分为面向架构、面向详细设计以及通用/需求三类，每种有明确的用途和负责人：

面向架构 (Architecture-Oriented)	面向详细设计 (Detail-Oriented)
README_ArchDesign.md — 模块分解、依赖、权衡	README_DetailDesign.md — 类设计、API 签名
README_UsageDesign.md — 公共边界、CLI/API 契约	README_StateDesign.md — 状态机、生命周期、并发
README_ErrorDesign.md — 容错、故障安全状态	
README_ResourceDesign.md — 资源分配、内存/CPU 预算	
README_PerfDesign.md — 性能预算、延迟限制	
README_CompatDesign.md — 兼容性矩阵、平台版本	
README_DiagnosisDesign.md — 可观测性、日志、遥测	

面向架构 (Architecture-Oriented)

面向详细设计 (Detail-Oriented)

README_VerifyDesign.md — 验证拓扑、测试策略

在面向架构和面向详细设计的表格之外，通用/需求组还包含 `README.md`（项目概览与主 SPEC 目录）、`README_UserStories.md`（强制性的项目级 TODO/DONE 故事账本，含 AC 追踪状态）以及 `README_UserGuide.md`（面向用户的用法指南）。这些文档先由开发者创建，之后由 `SPEC_updateUserStory` 和 `SPEC_reviewUserStory` 更新。

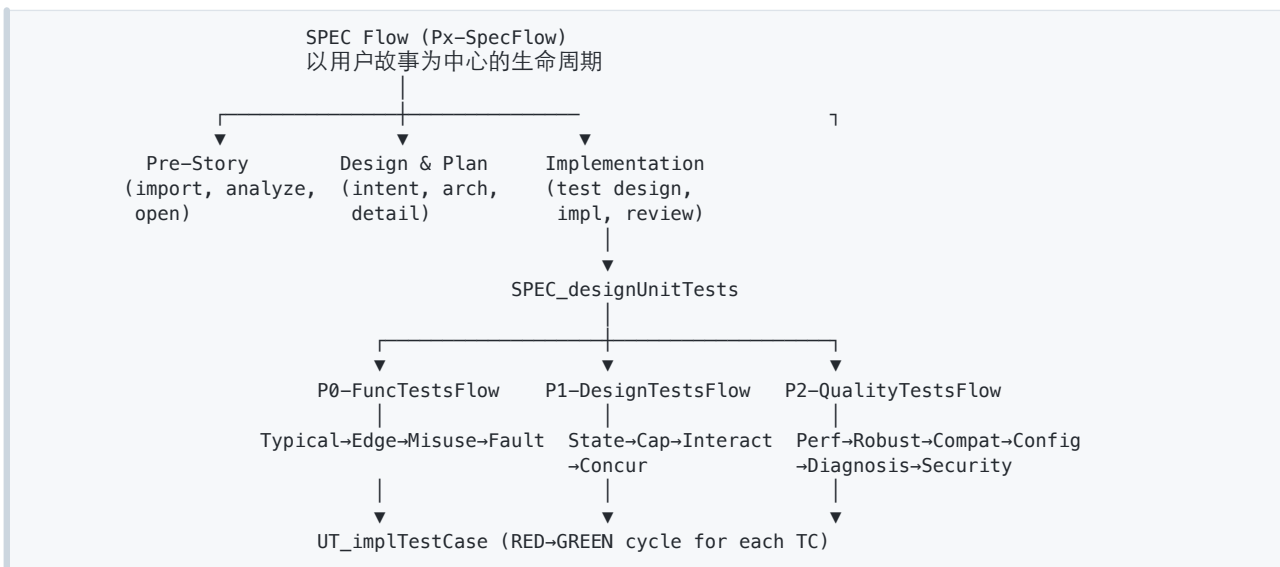
这些不是在初始化时一次性生成的模板——它们由 SPEC 命令按需创建，仅在项目需要该维度的文档时生成。

6. 按命令的模型层级指导。每个 SPEC 命令都分配了一个模型层级——SOTA reasoning 用于架构决策，High Performance 用于多制品推理和审查，Flash Speed 用于确定性的生命周期移动。这避免了在简单任务上过度支付计算开销，也避免了在架构级别的决策上投入不足。

UT 流程 — CaTDD 的嵌入式验证引擎

这是决定性的差异化因素。Spec Kit 和 OpenSpec 都没有测试方法论。它们描述构建什么以及按什么顺序，止步于实现阶段。CaTDD 走得更远：它将完整的、按类别驱动的 TDD 方法论嵌入到规格生命周期中。

双流架构



当 `SPEC_designUnitTests` 运行时，它生成的不是“编写测试”这样的通用任务，而是路由到 CaTDD 的 UT 流程中——P0 功能验证、P1 面向设计的测试、P2 质量属性。每个流程有其自己的命令序列、审查关卡和 Design Source Gate（设计源关卡）要求。

Interaction (P1) 以及 Diagnosis/Security (P2) 目前没有专门的 `UT_design*Skeleton` 命令：它们的流程直接通过对应的 `CaTDD_methodPrompt4Cat-*.md` method prompt 路由，在相同的 Discovery Gate 与审查契约下，将源关联的 US/AC/TC 写入规范的类别文件。在这些生命周期流程之外，`slashCommands` 还承载了一个运维性的 `HARNESS_*` 命令族（Px-HarnessKits），用于维护 harness，且绝不移动 SpecFlow 生命周期状态。

UT 流程提供的、其他规格工具都不具备的能力

1. 按类别驱动测试设计。15 种测试类别，每种有其自己的 method prompt（`CaTDD_methodPrompt4Cat-*.md`），定义位置、使用时机/避免时机路由规则、测试点、设计骨架契约和检查清单；大多数 prompt 还带有命名示例与常见错误指引。一个设计 Typical 骨架的 CodeAgent 阅读 `CaTDD_methodPrompt4Cat-Typical.md` 后，会精确知道应用什么模式、词汇和约束。
2. 嵌入测试文件的 US/AC/TC 可追溯性。规格说明不存在于单独的文档中。User Story、Acceptance Criteria 和 Test Case 规格以结构化注释的形式存在于与测试代码相同的文件中。当规格变化时，注释随之变化。当注释变化时，LLM 重新生成测试。不存在规格与代码之间的间隙。
3. P1 和 P2 的 Design Source Gate（设计源关卡）。在任何 P1 或 P2 骨架起草之前，必须存在所需的项目根设计文档：

P1 类别	所需源文件
State	<code>README_StateDesign.md</code> 或 <code>README_ArchDesign.md</code> 中的 <code>State Design</code> 章节
Capability	<code>README_DetailDesign.md</code>
Interaction	<code>README_ArchDesign.md</code> 中的顺序、交互或协作章节
Concurrency	<code>README_ResourceDesign.md</code>

P2 采用同样的关卡，在任何骨架起草之前：

P2 类别	所需源文件
Performance	<code>README_PerfDesign.md</code>
Robust	<code>README_ErrorDesign.md</code>
Compatibility	<code>README_CompatDesign.md</code>
Configuration	<code>README_DetailDesign.md</code>
Diagnosis	<code>README_DiagnosisDesign.md</code> / <code>README_VerifyDesign.md</code> 证据要求
Security	SecurityDesign 文档、威胁模型或安全策略

如果设计源缺失，UT 命令或直接 method prompt 路由停止并请求开发者——不做猜测，不凭空编造架构决策。

4. 集成到生命周期的 TDD RED→GREEN 纪律。每个测试用例都有状态标记：

○ TODO/PLANNED → ● RED/FAILING → ● GREEN/PASSED

`SPEC_implUnitTests` 命令调用 `UT_implTestCase`，遵循严格的循环：编写测试（RED），实现最小化生产代码（GREEN），审查（TRACE）。故事在所有必需的 TC 都为 GREEN 之前不能关闭。

5. 各层的质量关卡。直到所有 P0 测试为 GREEN，你不得从 P0 推进到 P1。直到架构验证通过（无死锁、无竞态条件、ThreadSanitizer 干净），你不得从 P1 推进到 P2。每个关卡是开发者在继续之前审查并批准的检查点。

三向对比

维度	GitHub Spec Kit	OpenSpec	CaTDD Px-SpecFlow
重心	文档 (spec.md, plan.md)	变更 (openspec/changes/)	用户故事 (生命周期状态)
生命周期状态	3 种隐式 (spec → plan → tasks)	3 种显式 (propose → apply → archive)	7 种显式 (pending → analyzed → todo → doing → suspend/abort → done)
中止机制	无 (手动关闭 issue/PR)	无 (手动删除变更文件夹)	<code>SPEC_abortUserStory</code> — 保留在 <code>abortUS/</code> ，有重新分析路径
意图对齐	自由形式的澄清对话	自由形式的对话	相互意图契约 (<code>SPEC_clearStoryIntent</code>)
测试方法论	无嵌入 (仅检查清单)	无嵌入	嵌入的 CaTDD — P0/P1/P2 UT 流程，15 种类别
规格-代码差距	spec.md → plan.md → code (3 个文档)	proposal.md → design.md → code (3 个文档)	无差距 — US/AC/TC 注释与测试代码存在于同一文件
TDD 集成	不存在	不存在	RED→GREEN 循环，配  →  →  状态标记
注释鲜活的设计	否 — 文档与代码分离	否 — 文档与代码分离	是 — 设计骨架存在于测试文件中
优先级框架	无	无	P0(核心)→P1(设计)→P2(质量)→P3(附加)
设计源关卡	无	无	是 — P1/P2 要求确认的 <code>README_*</code> 文档
模型层级指导	不存在	推荐高推理模型	按命令层级映射 (SOTA/HighPerf/FIash)
审查关卡	<code>/speckit.analyze</code> (跨制品)	<code>/opsx:verify</code> (实现后)	6 个审查关卡：架构→详细→故事→测试→代码→提交
制品持久化	文件系统 (specs 目录)	文件系统 (changes 目录)	团队共享的 <code>.catdd/spec/</code> ，每种制品有提交策略
项目根文档	Spec + plan，然后手动	Proposal + design，然后手动	13 种 README* SPEC 文档类型，通过 SPEC 命令按需创建
既有代码库 (Brownfield)	主要是全新项目 (green field)	优先既有代码库 (为现有代码库设计)	两者皆可 — 将现有代码分析为 US/AC/TC
安装方式	<code>uv tool install specify-cli</code> (Python)	<code>npm install -g @fission-ai/openspec</code> (TypeScript)	Markdown prompt 文件 (agent 无关)
Agent 集成	30+	25+	Copilot, Cline, Continue, utCodeAgentCLI
定制化	Extensions + presets + overrides	社区 schemas	<code>methodPrompts/</code> 为真理之源， <code>slashCommands/</code> 为命令层， <code>agentSkills/</code> 为包

为什么 CaTDD 是 BEYOND 的

Spec Kit 和 OpenSpec 都是有价值的工具，它们解决了规格优先的问题，但只解决了一半。

Spec Kit 给你一个结构化的流水线：constitution → spec → plan → tasks → implement。它告诉你构建什么，按什么顺序，用什么技术。但当实现完成时，你只有代码——除了检查清单，没有系统性的正确性证明。

OpenSpec 给你流动的变更管理：propose → apply → archive。它告诉你什么变了，设计是什么，运行什么任务。但归档后，你有一个已完成的变更——但没有嵌入式的验证来证明该变更满足自己的验收标准。

CaTDD Px-SpecFlow 两者都给你——而且走得更远：

1. 它管理用户故事的全生命周期，从原始导入到已验证闭环，并在范围或假设错误时提供显式的中止路径。
2. 它在设计开始前对齐人类与 LLM 的意图，避免了 LLM 驱动开发中最昂贵的错误：完美地构建了错误的东西。
3. 它嵌入完整的测试方法论 (UT P0/P1/P2) 到规格生命周期中。每个用户故事的验收标准由按类别驱动的测试用例验证。每个测试用例驱动一个 RED→GREEN TDD 循环。每个 GREEN 测试证明故事的一个片段是正确的。
4. 它消除规格-代码间隙。规格说明以结构化注释 (@[US], @[AC], @[TC]) 存在于与测试代码相同的文件中。当代码变化时，注释随之变化。当注释变化时，测试随之变化。规格与验证不可分离。
5. 它按领域重要性分类验证。P0 测试你的核心域（使你的业务独特的内容）。P1 测试你的架构。P2 测试你的质量属性。这是 DDD 的战略设计应用于验证——它告诉 LLM 不仅要“测试这个”，还要“测试这个，因为它对业务至关重要”。

规格驱动堆栈

将它们看作规格驱动开发堆栈的层次：

CaTDD Px-SpecFlow + UT flows "证明它是正确的" 用户故事生命周期 + 嵌入式 TDD
Spec Kit / OpenSpec "决定构建什么以及如何构建" 以文档为中心或以变更为中心
AI Coding Agents (Copilot, Cline, Claude, ...) "生成代码"

Spec Kit 和 OpenSpec 运行在中间层——它们结构化构建什么。CaTDD 运行在顶层——它验证构建出来的东西是否正确。它们不是竞争对手，而是互补的。

但如果你只能选一个，必须在“知道构建什么”和“知道构建出来的东西是正确的”之间做出选择——CaTDD 给了你不同于任何单独的规格优先工具的验证保证。

Comments is Verification Design. LLM Generates Code. Iterate Forward Together.